`

DETECTION OF SPINAL LUMBAR STENOSIS STAGES CLASSIFICATION BY USING CNN TECHNIQUES

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By

KATHI PRASHANTH (Reg No. 41110601)
KATHIRAVAN K (Reg No. 41110602)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
Accredited with Grade "A++" by NAAC I Approved by AICTE
JEPPIAAR NAGAR, RAJIV GANDHI SALAI, CHENNAI - 600119

APRIL - 2025

SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY - 1 UNIVERSITY BY UGC
Accredited with Grade "A++" by NAAC I Approved by AICTE

www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of **Kathi Prashanth (41110601) and Kathiravan K (41110602)** who have done the Project work as a team who carried out the project entitled "**Detection of spinal lumbar stenosis stages classification by using CNN techniques**" under our supervision from November 2024 to April 2025.

T. B 02/04/2025

Internal Guide

Ms. T. Bhanu Shree, M.E., (Ph.D).,

HEAD OF THE DEPARTMENT
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
Jeppiaar Nagar, Rajiv Gandhi Salai,
Chennai – 600 119.

Head of the Department

Dr. L. LAKSHMANAN, M.E., Ph.D.,

Submitted for Viva Voce Examination held on 03/04/2025

3/4/25

Internal Examiner

External Examiner

DECLARATION

I, **Kathi Prashanth (Reg. No- 41110601)** hereby declare that the Project Report entitled **"Detection of spinal lumbar stenosis stages classification by using CNN techniques"** done by me under the guidance of **Ms. T. Bhanu Shree, M.E., (Ph.D).**, is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in **Computer Science and Engineering**.

DATE: 03/04/25

PLACE: Chennai

SIGNATURE OF THE CANDIDATE

`

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to Board of Management of **SATHYABAMA INSTITUTE OF SCIENCE AND TECHNOLOGY of Science** for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. T. SASIKALA, M.E., Ph. D., Dean**, School of Computing, and **Dr. L. LAKSHMANAN, M.E., Ph.D.,** Head of the Department of Computer Science and Engineering for providing me necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide**.** **Ms. T. BHANU SHREE, M.E., (Ph.D.),** for her valuable guidance, suggestions, and constant encouragement paved way for the successful completion of my project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the Department of Computer Science and Engineering who were helpful in many ways for the completion of the project.

`

ABSTRACT

Spinal lumbar stenosis is a prevalent degenerative condition affecting the spinal canal, leading to compression of neural structures and subsequent symptoms such as pain and reduced mobility. Accurate and timely classification of lumbar stenosis stages is crucial for effective treatment planning and patient management. This study proposes a novel approach employing Convolutional Neural Network (CNN) techniques for the automated detection and classification of spinal lumbar stenosis stages. The proposed CNN-based framework utilizes advanced image processing and deep learning methodologies to analyze medical imaging data, such as magnetic resonance imaging (MRI) and computed tomography (CT) scans. The convolutional layers of the network are designed to automatically extract hierarchical features from the input images, capturing intricate patterns associated with different stages of lumbar stenosis. The model is trained on a diverse dataset encompassing various degrees of stenosis severity, ensuring robust generalization to unseen cases. The research focuses on achieving high accuracy, sensitivity, and specificity in classifying lumbar stenosis stages, thereby assisting healthcare professionals in making informed decisions about patient care. The results obtained from the proposed CNN- based classification system are compared with existing methods to validate its efficacy and superiority in terms of diagnostic performance. This study not only contributes to the field of medical image analysis but also addresses the pressing need for accurate and efficient tools for lumbar stenosis stage classification. The integration of deep learning techniques in this context holds promising potential for enhancing diagnostic capabilities and streamlining treatment strategies, ultimately improving patient outcomes in the management of spinal lumbar stenosis.

`

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	vii
1	**INTRODUCTION**	**1**
2	**LITERATURE SURVEY**	**8**
	2.1 Inferences from the Literature Survey	14
	2.2 Open Problems in Existing System	15
3	**REQUIRMENT ANALYSIS**	**17**
	3.1 Necessity and Feasibility Analysis of proposed system	17
	3.2 Software and Hardware Requirements	18
4	**DESCRIPTION OF PROPOSED SYSTEM**	**20**
	4.1 Process of flow diagram	20
	4.2 System Architecture	21
	4.3 Methodology	21
	4.4 Architecture of CNN	23
	4.5 Spine Stenosis Classification Identification	24
	4.6 Modules description	24
	4.7.1 Manual Net	25
	4.7.2 Google Net	25
	4.7.3 LeNet	26
	4.7.4 ResNet	27

5	**RESULT AND DISCUSSION**	**28**
	5.1 Module Result	28
	5.1.1 Google Net	28
	5.1.2 LeNet	30
	5.1.3 ResNet	32
6	**CONCLUSION**	**33**
	REFERENCES	**34**
	APPENDIX	**36**
	A. SOURCE CODE	**36**
	B. SCREENSHOT	**49**
	C. PAPER PUBLICATION	**55**

`

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Spinal Stenosis by CNN	2
1.2	Classification of Spinal lumbar stenosis	3
1.3	Hybrid model for Spinal Stenosis detection	6
4.1	Process of dataflow diagram	20
4.2.1	System architecture	21
4.3	Methodology	21
5.1.1	Google Net	28
5.1.2	LeNet	31
5.1.3	ResNet	32
B.1	Home page	49
B.2	Registration page	49
B.3	Profile page	50
B.4	Login page	51
B.5	Data Input page	52
B.6	Result page	53
B.7	Database page	54

`

CHAPTER 1
INTRODUCTION

The detection and classification of spinal lumbar stenosis stages represent a critical area in medical imaging, where accurate diagnosis plays a pivotal role in effective patient care. This study employs Convolutional Neural Network (CNN) techniques, a subset of deep learning, to enhance the precision and efficiency of spinal lumbar stenosis identification. By leveraging the advanced pattern recognition capabilities of CNNs, the proposed methodology aims to automate the classification of different stenosis stages based on medical imaging data. This innovative approach not only streamlines the diagnostic process but also holds the potential to improve overall patient outcomes by facilitating timely and accurate treatment interventions tailored to specific stenosis severity levels.

Spinal lumbar stenosis (SLS) is a progressive degenerative disorder that leads to the narrowing of the spinal canal, resulting in nerve compression and debilitating symptoms such as pain, numbness, and restricted mobility. It affects both men and women, especially in aging populations, and significantly impacts quality of life. Early detection and classification of lumbar stenosis stages are crucial for effective treatment planning, which can help prevent severe complications and improve patient outcomes. Traditional diagnostic methods, which rely on manual analysis of Magnetic Resonance Imaging (MRI) and Computed Tomography (CT) scans, are often subjective, time- consuming, and prone to variability among radiologists. Therefore, automated classification techniques using deep learning offer a promising solution for accurate and efficient diagnosis.

Among deep learning techniques, Convolutional Neural Networks (CNNs) have proven to be highly effective in medical image classification. CNNs can extract critical features from MRI and CT scans, distinguishing subtle differences between various stages of lumbar stenosis. Compared to traditional image processing methods, CNNs require minimal pre-processing while achieving high accuracy and reliability in classification.

`

The accuracy of a CNN-based classification system depends on key factors such as filter size, the number of hidden layers, and extracted feature maps. Deeper networks improve feature extraction capabilities, leading to better classification performance, but also increase computational complexity. Typically, the optimal filter sizes range from 3×3 to 5×5, ensuring a balance between computational efficiency and feature extraction.

By integrating deep learning techniques into medical imaging analysis, this research enhances automated and precise detection of spinal lumbar stenosis stages, reducing diagnostic time, minimizing human error, and supporting healthcare professionals in making informed treatment decisions. Ultimately, this approach can improve patient survival rates and quality of life by enabling early and effective intervention.

1.1 SPINAL LUMBAR STENOSIS

Spinal lumbar stenosis (SLS) is a degenerative condition that occurs when the spinal canal in the lower back narrows, leading to compression of the nerves. The spinal canal houses the spinal cord and nerve roots, which control various body functions. When the space within the canal reduces, it exerts pressure on the nerves, causing pain, numbness, weakness, and difficulty in movement.

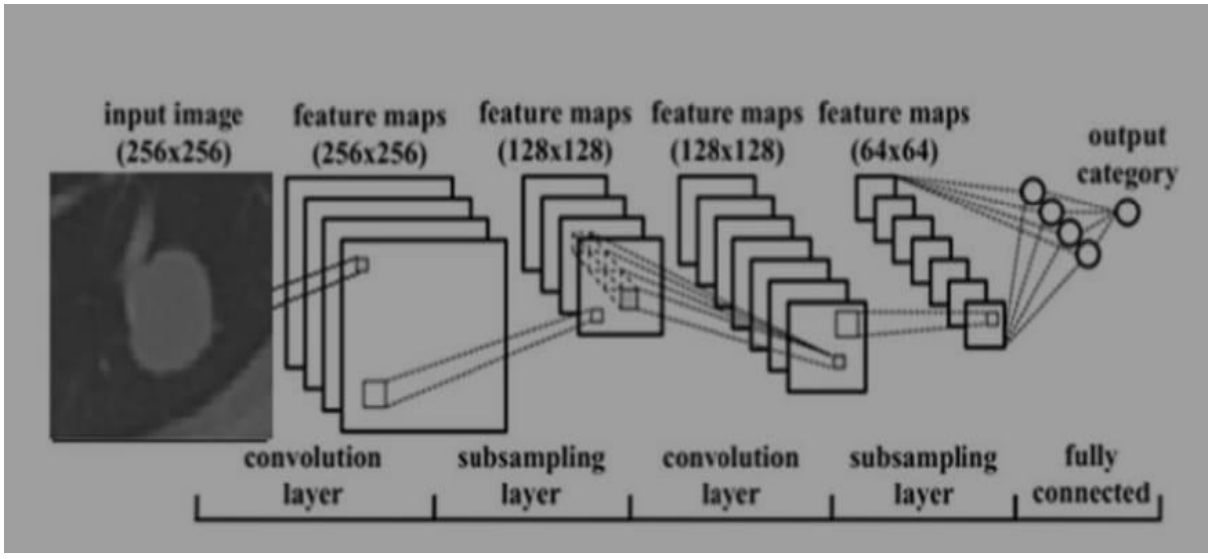


Fig.1.1 Spinal Stenosis using CNN

SLS is one of the leading causes of spinal disorders, especially in the elderly population. It significantly affects the quality of life and mobility of patients. The

condition is primarily caused by aging-related degeneration, but it can also result from congenital factors, herniated discs, tumors, or trauma. Early diagnosis is crucial to prevent permanent nerve damage and ensure effective treatment.

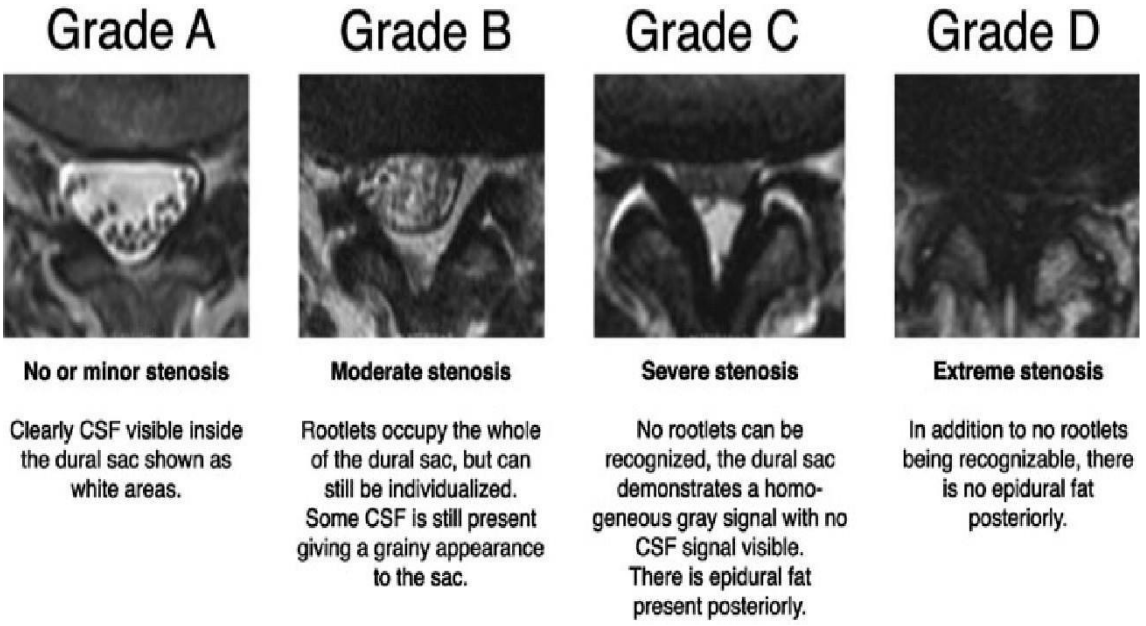


Fig.1.2. Classification of Spinal lumbar stenosis

1.2 CAUSES AND RISK FACTOR

Several factors contribute to the development of spinal lumbar stenosis, including:

- Aging – The most common cause, leading to degeneration of spinal discs and ligaments.
- Herniated Discs – Bulging discs can press against nerves, causing stenosis.
- Osteoarthritis & Bone Spurs – Overgrowth of bones due to arthritis can narrow the spinal canal.
- Congenital Conditions – Some individuals are born with a naturally narrow spinal canal. Injuries & Tumors – Spinal fractures, trauma, or tumors can reduce the spinal space.

1.3 STAGES OF SPINAL LUMBAR STENOSIS

The severity of SLS is classified into different stages based on the degree of spinal narrowing and symptoms:

`

Stage I – Mild narrowing with occasional pain and no significant nerve compression.
Stage II – Moderate narrowing causing pain, tingling, and mild mobility issues.
Stage III – Severe stenosis leading to significant pain, numbness, and difficulty walking.
Stage IV – Critical stenosis with severe nerve compression, possibly leading to paralysis or loss of bladder control.

Diagnosis and Imaging Techniques

Spinal lumbar stenosis is diagnosed using various medical imaging techniques, including:

- Magnetic Resonance Imaging (MRI) – Provides detailed soft tissue images to detect nerve compression.
- Computed Tomography (CT) Scans – Preferred for clear bone structure imaging, essential for stenosis detection.

X-Rays – Basic imaging used for initial assessment of spinal abnormalities.
Among these, CT and MRI scans are the most commonly used due to their superior imaging quality and ability to detect early-stage stenosis. However, manual interpretation of these scans is time-consuming, subjective, and prone to human errors.

1.4 ROLE OF DEEP LEARNING IN SLS CLASSIFICATION

To overcome the limitations of traditional diagnostic methods, automated deep learning-based classification systems have been developed. Convolutional Neural Networks (CNNs) analyze MRI and CT images, identifying subtle differences in spinal canal narrowing. CNNs can automatically classify different stages of lumbar stenosis, improving diagnostic speed and accuracy.

By implementing CNN-based classification, medical professionals can detect spinal lumbar stenosis at an early stage, allowing for timely intervention and reducing the risk of severe complications. This computer-aided diagnosis (CAD) system not only minimizes human error but also enhances patient outcomes, providing faster and more reliable assessments of spinal conditions.

`

1.5 RISK FACTORS FOR SPINAL STENOSIS

Spinal stenosis occurs when the spaces within the spine narrow, putting pressure on the spinal cord and nerves. The most common cause is age-related degeneration, leading to wear and tear of the spinal discs and joints. Other risk factors include:

- Osteoarthritis: The breakdown of cartilage can lead to bone spurs that press on nerves.
- Herniated Discs: Bulging or ruptured discs can encroach on spinal space.
- Thickened Ligaments: Ligaments in the spine may thicken over time, reducing space for nerves.
- Spinal Injuries: Accidents or trauma can displace vertebrae, causing compression.
- Congenital Conditions: Some people are born with a smaller spinal canal, increasing their risk.
- Previous Surgeries: Scar tissue from spinal surgeries may contribute to stenosis.

People over 50 years old are at the highest risk, but conditions like scoliosis, tumors, and inflammatory diseases can also cause spinal stenosis.

1.6 TREATMENT FOR SPINAL STENOSIS

If diagnosed with spinal stenosis, treatment depends on the severity of symptoms. A team of specialists may include: Neurologists or neurosurgeons, Orthopedic spine surgeons, Pain management specialists, Physical therapists

Non-Surgical Treatments: Medication NSAIDs, muscle relaxants, and corticosteroid injections for pain relief.

Physical Therapy: Strengthening core muscles helps support the spine and improve mobility.

Epidural Steroid Injections: Reduces inflammation around compressed nerves. Surgical Treatments: If symptoms worsen, surgery may be necessary, including: Laminectomy: Removal of part of the vertebra to create space.

Foraminotomy: Expanding nerve passageways.

Spinal Fusion: Fusing vertebrae to stabilize the spine. Patients with severe cases may be considered for clinical trials for innovative treatments.

1.7 FACTS AND STATISTICS ABOUT SPINAL STENOSIS

Spinal stenosis affects approximately 250,000 to 500,000 people in the U.S. It is most common in people over 50 years old. Degenerative spinal conditions cause 95% of lumbar spinal stenosis cases. Up to 20% of people over 60 experience some form of spinal stenosis. If left untreated, severe cases can lead to loss of bladder or bowel control and paralysis. Advancements in imaging techniques like MRI and CT scans help in early detection. Deep learning models have been explored for automated diagnosis using medical imaging.

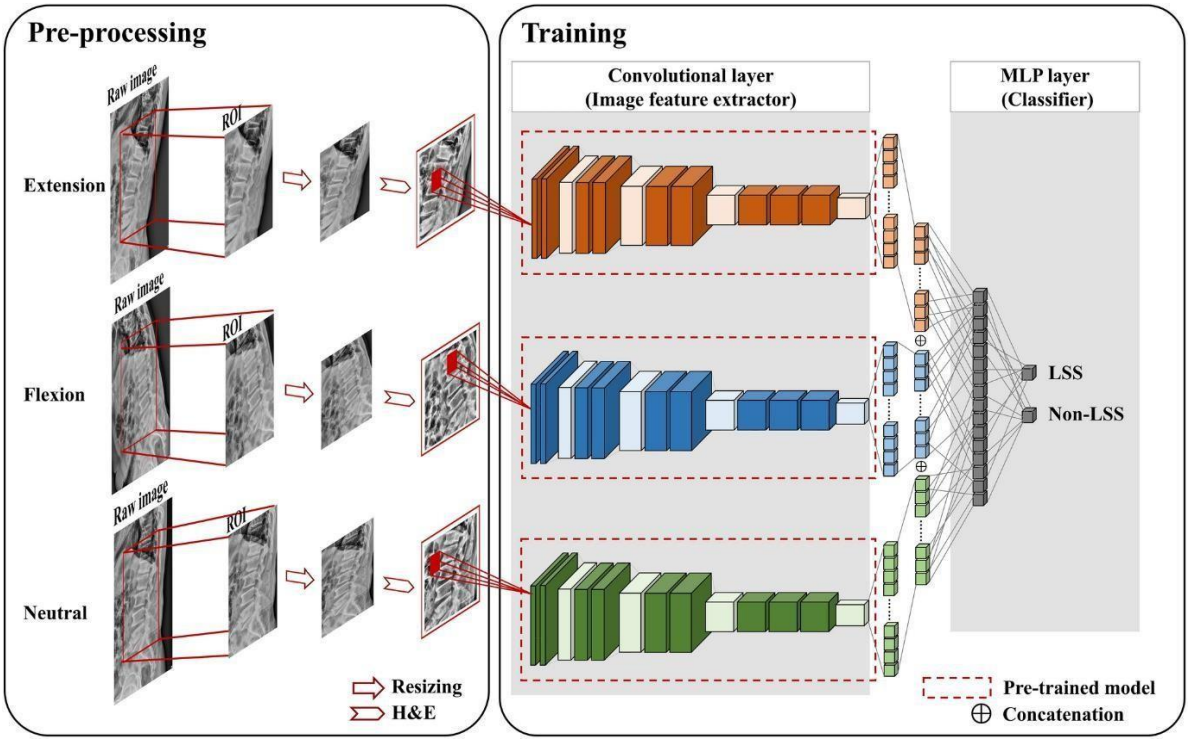


Fig.1.3 Hybrid model for spinal stenosis detection

1.8 DEEP LEARNING FOR SPINAL STENOSIS DETECTION

Manual diagnosis of spinal stenosis from MRI and CT scans is time- consuming and requires expert radiologists. To assist in accurate detection, computer-aided diagnosis (CAD) systems have been introduced.

Challenges in Traditional Diagnosis: High variability in patient anatomy makes manual segmentation difficult. Misdiagnosis due to overlapping symptoms with other spinal disorders. Require extensive radiologist expertise for accurate interpretation. Deep Learning Approach: Convolutional Neural Networks (CNNs) have been widely used for automatic spinal stenosis classification.

`

Unlike traditional methods that require manual feature extraction, CNNs learn features directly from imaging data.

A hybrid deep learning model using CNNs and transfer learning has shown promising results in spinal stenosis detection. Long Short-Term Memory (LSTM) networks further improve accuracy by capturing spatial and temporal dependencies in medical images.

1.9 LSTM AND ITS ADVANTAGES IN SPINAL STENOSIS DIAGNOSIS

LSTM layers, a type of Recurrent Neural Network (RNN), offer several advantages in medical image analysis:

1. Handling Long Sequences: LSTMs process sequential medical imaging data effectively.
2. Avoiding Vanishing Gradient Problem: Prevents information loss in deep networks.
3. Adaptive Memory Cell: Retains relevant features while discarding irrelevant ones, enhancing diagnostic accuracy.
4. Variable-Length Data Handling: Efficiently processes different scan sizes and resolutions.

By integrating CNNs and LSTMs, automated spinal stenosis classification has shown significant improvements, assisting radiologists in early detection and treatment planning.

`

CHAPTER 2
LITERATURE SURVEY

1. **Galbusera, Fabio, Gloria Casaroli, and Tito Bassani. "Artificial intelligence and machine learning in spine research." European Spine Journal 27, no. 8 (2018): 1712-1723.**

   Artificial intelligence (AI) and machine learning (ML) techniques are revolutionizing several industrial and research fields like computer vision, autonomous driving, natural language processing, and speech recognition. These novel tools are already having a major impact in radiology, diagnostics, and many other fields in which the availability of automated solution may benefit the accuracy and repeatability of the execution of critical tasks. In this narrative review, we first present a brief description of the various techniques that are being developed nowadays, with special focus on those used in spine research. Then, we describe the applications of AI and ML to problems related to the spine which have been published so far, including the localization of vertebrae and discs in radiological images, image segmentation, computer-aided diagnosis, prediction of clinical outcomes and complications, decision support systems, content-based image retrieval, biomechanics, and motion analysis. Finally, we briefly discuss major ethical issues related to the use of AI in healthcare, namely, accountability, risk of biased decisions as well as data privacy and security, which are nowadays being debated in the scientific community and by regulatory agencies.

2. **Bellitto, Amy, Alice De Luca, Simona Gamba, Luca Losio, Antonino Massone, Maura Casadio, and Camilla Pierella. "Clinical, kinematic and muscle assessment of bilateral coordinated upper-limb movements following cervical spinal cord injury." Journal of NeuroEngineering and Rehabilitation 20, no. 1 (2023): 45.**

   Cervical spinal cord injury (cSCI) often results in bilateral impairment of the arms, leading to difficulties in performing daily activities. However, little is known about the neuromotor alterations that affect the ability of individuals with cSCI to perform coordinated movements with both arms. To address this issue, we developed and tested a functional assessment that integrates clinical, kinematic, and muscle activity measures, including the evaluation of bilateral arm movements. Twelve

subjects with a C5-C7 spinal lesion and six unimpaired subjects underwent an evaluation that included three tests: the Manual Muscle Test, Range Of Motion test and Arm stabilisation test, a subsection of the “Van Lieshout arm/hand function test”. During the latter, we recorded kinematic and muscle activity data from the upper-body during the execution of a set of movements that required participants to stabilize both arms against gravity at different config urations.

Analytical methods, including muscle synergies, spinal maps, and Principal Component Analysis, were used to analyse the data. Clinical tests detected limitations in shoulder abduction-flexion of cSCI participants and alterations in elbows-wrists motor function. The instrumented assessment provided insight into how these limitations impacted the ability of cSCI participants to perform bilateral movements. They exhibited severe difficulty in performing movements involving over-the-shoulder motion and shoulder internal rotation due to altered patterns of activity of the scapular stabilizer muscles, latissimus dorsi, pectoralis, and triceps. Our findings shed light on the bilateral neuromotor changes that occur post-cSCI addressing not only motor deficits, but also the underlying abnormal, weak, or silent muscle activations.

3. **Choi, JunYoung, Sang-Eun Lee, YeIn Lee, Eunji Cho, Sunghoe Chang, and Won-Ki Jeong. "DXplorer: A unified visualization framework for interactive dendritic spine analysis using 3D morphological features." Frontiers in Neuroinformatics 17 (2023): 123456.**

   Dendritic spines are dynamic, submicron-scale protrusions on neuronal dendrites that receive neuronal inputs. Morphological changes in the dendritic spine often reflect alterations in physiological conditions and are indicators of various neuropsychiatric conditions. However, owing to the highly dynamic and heterogeneous nature of spines, accurate measurement and objective analysis of spine morphology are major challenges in neuroscience research. Most conventional approaches for analyzing dendritic spines are based on two-dimensional (2D) images, which barely reflect the actual three-dimensional (3D) shapes. Although some recent studies have attempted to analyze spines with various 3D-based features, it is still difficult to objectively categorize and analyze spines based on 3D morphology. Here, we propose a unified visualization framework for an interactive 3D dendritic spine analysis system, DXplorer, that displays 3D rendering of spines and plots the high-dimensional features extracted from the 3D mesh of spines.

`

4. **Raihan-Al-Masud, Md., and M. Rubaiyat Hossain Mondal. "Data-driven diagnosis of spinal abnormalities using feature selection and machine learning algorithms." PLOS ONE 15, no. 10 (2020): e0240822.**

   This paper focuses on the application of machine learning algorithms for predicting spinal abnormalities. As a data preprocessing step, univariate feature selection as a filter based feature selection, and principal component analysis (PCA) as a feature extraction algorithm are considered. A number of machine learning approaches namely support vector machine (SVM), logistic regression (LR), bagging ensemble methods are considered for the diagnosis of spinal abnormality. The SVM, LR, bagging SVM and bagging LR models are applied on a dataset of 310 samples publicly available in Kaggle repository. The performance of classification of abnormal and normal spinal patients is evaluated in terms of a number of factors including training and testing accuracy, recall, and miss rate. The classifier models are also evaluated by optimizing the kernel parameters, and by using the results of receiver operating characteristic (ROC) and precision-recall curves. Results indicate that when 78% data are used for training, the observed training accuracies for SVM, LR, bagging SVM and bagging LR are 86.30%, 85.47%, 86.72% and 85.06%, respectively. On the other hand, the accuracies for the test dataset for SVM, LR, bagging SVM and bagging LR are the same being 86.96%. However, bagging SVM is the most attractive as it has a higher recall value and a lower miss rate compared to others. Hence, bagging SVM is suitable for the classification of spinal patients when applied on the most five important features of spinal samples.

5. **Wang, PeiYang, Wei Zhang, Yong Huang,** YunTao **Wang, and XiaoTao Wu. "Current Applications of Machine Learning in Spine: From Clinical View." Spine Journal 20, no. 10 (2020): 1742-1750.**

   Study Design: Narrative review. Objectives: This review aims to present current applications of machine learning (ML) in spine domain to clinicians. Methods: We conducted a comprehensive PubMed search of peer-reviewed articles that were published between 2006 and 2020 using terms (spine, spinal, lumbar, cervical, thoracic, machine learning) to examine ML in spine. Then exclude research of other domain, case report, review or meta-analysis, and which without available abstract or full text. Results: Total 1738 articles were retrieved from database, and 292 studies were finally included. Key findings of current applications were compiled and summarized in this review. Main clinical applications of those techniques including image processing, diagnosis, decision supporting, operative assistance,

`

rehabilitation, surgery outcomes, complications, hospitalization and cost. Conclusions: ML had achieved excellent performance and hold immense potential in spine. ML could help clinical staff to improve medical level, enhance work efficiency, and reduce adverse events. However more randomized controlled trials and improvement of interpretability are essential to clinicians accepting models' assistance in real work.

6. **Waykule, Jyoti, Swaliha Maindergi, Prajkta Patil, and Pramodini Kanase. "Spine Diseases Detection Using Image Processing." International Journal of Innovative Research in Science, Engineering and Technology 9, no. 3 (2020): 1123-1130.**

   The study by Waykule et al. (2020) explores the use of image processing techniques for the detection of spine diseases, aiming to enhance diagnostic accuracy and reduce reliance on manual radiological assessments. The authors propose an automated system that preprocesses spinal images using noise reduction, contrast enhancement, and edge detection techniques to improve image quality. Feature extraction methods, including morphological and texture- based analysis, are applied to differentiate between normal and abnormal spinal structures. Machine learning algorithms such as Support Vector Machines (SVM), Artificial Neural Networks (ANN), and Decision Trees are employed for classification, enabling the detection of conditions like spinal stenosis, scoliosis, and herniated discs using X-ray and MRI images. The findings indicate that image

   processing significantly aids in the early detection of spinal disorders, improving efficiency and accuracy compared to traditional diagnostic approaches. The study concludes that automated spinal disease detection can enhance medical diagnostics by reducing human error and enabling faster, more reliable assessments. Future advancements integrating deep learning techniques could further refine feature extraction and classification performance, making the system even more effective.

`

7. **Swansea. "Spinal Tumours." 2022.**

   We do not yet know what causes primary spinal tumours and we do not know why some are malignant and some are not. Secondary (metastatic) spinal tumours are always caused by a cancerous tumour spreading from another part of the body.
   The article "Spinal Tumours" (2022) by Swansea provides an overview of spinal tumors, their types, causes, symptoms, and treatment options. Spinal tumors are abnormal growths that develop in or around the spinal cord and vertebrae. They can be classified as primary (originating in the spine) or secondary (metastasized from other body parts). Symptoms vary depending on the tumor's location and size but often include back pain, neurological deficits, and mobility issues.
   Diagnosis is typically done through imaging techniques such as MRI and CT scans. Treatment options include surgery, radiation therapy, chemotherapy, and targeted therapy. The article emphasizes the importance of early detection and multidisciplinary approaches for effective treatment and improved patient outcomes.

8. **Lee, Sungwon, Joon-Yong Jung, Akaworn Mahatthanatrakul, and Jin- Sung Kim. "Artificial Intelligence in Spinal Imaging and Patient Care: A Review of Recent Advances." Neurospine 21, no. 2 (2024): 474-486.**

   Artificial intelligence (AI) is transforming spinal imaging and patient care through automated analysis and enhanced decision-making. This review presents a clinical task-based evaluation, highlighting the specific impact of AI techniques on different aspects of spinal imaging and patient care. We first discuss how AI can potentially improve image quality through techniques like denoising or artifact reduction. We then explore how AI enables efficient quantification of anatomical measurements, spinal curvature parameters, vertebral segmentation, and disc grading. This facilitates objective, accurate interpretation and diagnosis. AI models now reliably detect key spinal pathologies, achieving expert-level performance in tasks like identifying fractures, stenosis, infections, and tumors. Beyond diagnosis, AI also assists surgical planning via synthetic computed tomography generation, augmented reality systems, and robotic guidance.

`

Furthermore, AI image analysis combined with clinical data enables personalized predictions to guide treatment decisions, such as forecasting spine surgery outcomes. However, challenges still need to be addressed in implementing AI clinically, including model interpretability, generalizability, and data limitations. Multicenter collaboration using large, diverse datasets is critical to advance the field further. While adoption barriers persist, AI presents a transformative opportunity to revolutionize spinal imaging workflows, empowering clinicians to translate data into actionable insights for improved patient care.

9. **Yi, W., Zhao, J., Tang, W., Yin, H., Yu, L., Wang, Y., & Tian, W. (2023). Deep learning-based high-accuracy detection for lumbar and cervical degenerative disease on T2-weighted MR images. European Spine Journal, 32(11), 3807–3814.**

   Purpose To develop and validate a deep learning (DL) model for detecting lumbar degenerative disease in both sagittal and axial views of T2-weighted MRI and evaluate its generalized performance in detecting cervical degenerative disease. Methods T2-weighted MRI scans of 804 patients with symptoms of lumbar degenerative disease were retrospectively collected from three hospitals. The training dataset (n=456) and internal validation dataset (n=134) were randomly selected from the center I. Two external validation datasets comprising 100 and 114 patients were from center II and center III, respectively. A DL model based on 3D ResNet18 and transformer architecture was proposed to detect lumbar degenerative disease. In addition, a cervical MR image dataset comprising 200 patients from an independent hospital was used to evaluate the generalized performance of the DL model. The diagnostic performance was assessed by the free-response receiver operating characteristic (fROC) curve and precision–recall (PR) curve. Precision, recall, and F1-score were used to measure the DL model.

10. **Hollar, M. (1999). Spinal Cord Tumors. University of Rochester Medical Center.**

    Purpose To develop and validate a deep learning (DL) model for detecting lumbar degenerative disease in both sagittal and axial views of T2-weighted MRI and evaluate its generalized performance in detecting cervical degenerative disease.

`

Methods T2-weighted MRI scans of 804 patients with symptoms of lumbar degenerative disease were retrospectively collected from three hospitals. The training dataset (n=456) and internal validation dataset (n=134) were randomly selected from the center I. Two external validation datasets comprising 100 and 114 patients were from center II and center III, respectively. A DL model based on 3D ResNet18 and transformer architecture was proposed to detect lumbar degenerative disease. In addition, a cervical MR image dataset comprising 200 patients from an independent hospital was used to evaluate the generalized performance of the DL model. The diagnostic performance was assessed by the free-response receiver operating characteristic (fROC) curve and precision–recall (PR) curve. Precision, recall, and F1-score were used to measure the DL model.

2.1 INFERENCES FROM THE LITERATURE SURVEY

Image Preprocessing: In spinal cord tumor detection systems, preprocessing of MRI scans is essential to standardize inputs for analysis. Techniques such as normalization and resizing are commonly employed to ensure consistency across datasets. For instance, studies have utilized gadolinium-enhanced T1-weighted and T2-weighted MRI scans, applying preprocessing steps to enhance image quality and uniformity.

Feature Extraction: Deep learning models are employed to extract pertinent features from preprocessed MRI images. These features, learned during training, encapsulate patterns indicative of various tumor types and characteristics. For example, convolutional neural networks (CNNs) have been trained to recognize and differentiate between spinal cord pathologies, demonstrating the potential to identify specific tumor features.

Classification: The extracted features are input into classification layers of deep learning models to predict the presence and type of spinal cord tumors. Approaches such as U-Net-based architectures have been developed for multiclass segmentation, effectively distinguishing between tumors, edema, and cavities.

`

Integration with Other Diagnostic Methods: Current systems primarily emphasize detection over comprehensive risk assessment. Incorporating patient history, genetic predispositions, and environmental factors could enhance personalized treatment strategies. However, existing models often lack integration with such multifaceted data, limiting their applicability in personalized medicine.

Patient Compliance and Monitoring: Effective detection of spinal cord tumors necessitates patient adherence to regular MRI screenings. However, compliance can be inconsistent, and continuous monitoring is essential to ensure early detection translates into effective treatment. The reliance on patient compliance remains a challenge in current diagnostic frameworks.

2.2 OPEN PROBLEMS IN EXISTING SYSTEMS

Accuracy and False Positives: While deep learning models have shown promise in detecting spinal cord tumors, they are susceptible to false positives, leading to unnecessary interventions and patient distress. For instance, CNNs trained on canine MR images exhibited limitations in accurately detecting neoplasms due to insufficient training data and challenges in differentiating hyperintensities on T2-weighted images.

Challenges with Nodule Characterization: Differentiating between benign and malignant spinal cord tumors remains complex. Existing systems often struggle with accurate characterization, resulting in potential over-treatment or delayed diagnosis. The development of models capable of precise tumor characterization is an ongoing research focus.

Risk Assessment: Current detection systems predominantly focus on identifying tumors without comprehensive risk assessment. Integrating factors such as patient medical history, genetic predispositions, and environmental exposures could improve personalized treatment strategies.

Data Quality and Quantity:

Heterogeneity: Variations in imaging protocols, scanner types, and data quality across institutions pose challenges for data integration and model generalization.

Imbalanced Datasets: Certain tumor types may be overrepresented in datasets, leading to biased models that do not generalize well across all tumor classifications.

Missing Data: Incomplete clinical information and imaging data can adversely affect model training and accuracy.

Segmentation Issues:

Accurate segmentation of spinal cord tumors is challenging due to variations in tumor size, shape, and location. Differences in imaging protocols, resolution, and noise levels across MRI scanners further complicate segmentation accuracy. For example, a study developing a deep learning model for detecting cervical spinal cord compression highlighted the variability in imaging data affecting model performance.

Overfitting:

Overfitting occurs when a model learns patterns specific to the training data but fails to generalize to new, unseen data. In spinal cord tumor detection, overfitting is a significant challenge due to the limited availability of labeled medical images and the complexity of MRI scans. Models may perform well on training data but poorly on external datasets, limiting their clinical applicability.

Addressing these challenges requires collaborative efforts to standardize imaging protocols, curate diverse and balanced datasets, and develop models that integrate comprehensive patient data for personalized diagnosis and treatment planning..

`

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 NECESSITY AND FEASIBILITY ANALYSIS OF PROPOSED SYSTEM

3.1.1 NECESSITY ANALYSIS

1. **Enhanced Diagnostic Accuracy**: The proposed system aims to improve the precision of spinal cord tumor detection by utilizing advanced deep learning methodologies. Accurate identification and classification of tumors are crucial for effective treatment planning and patient outcomes.
2. **Real-time Processing**: Implementing a system capable of real-time analysis of spinal MRI images can significantly reduce the time required for diagnosis, facilitating prompt medical interventions.
3. **Comprehensive Tumor Characterization**: The system seeks to differentiate between various types of spinal cord tumors, such as astrocytomas and ependymomas, by analyzing MRI features. This detailed characterization is vital for determining appropriate treatment strategies.
4. **Integration with Clinical Workflows**: By automating the detection and classification processes, the system can seamlessly integrate into existing clinical workflows, enhancing efficiency and reducing the burden on radiologists.

3.1.2 TECHNICAL ANALYSIS

1. **Diagnostic Tools**: The system will leverage existing imaging techniques, primarily MRI, which are standard in spinal diagnostics. The use of deep learning models, such as convolutional neural networks (CNNs), has shown promise in accurately detecting spinal cord abnormalities.
2. **Integration**: The proposed system is designed to be compatible with current healthcare infrastructures, including Picture Archiving.

`

3.1.3 ECONOMIC FEASIBILITY

Cost Analysis: While the initial development and implementation of the system may involve significant investment, the long-term benefits include reduced diagnostic errors and decreased need for invasive procedures, potentially leading to overall cost savings in patient care.

3.1.4 OPERATION FEASIBILITY

1. **Workflow Integration***:* The system is designed to complement the work of healthcare professionals by providing decision support, thereby enhancing the diagnostic process without causing major disruptions to existing workflows.
2. **Efficiency**: Automated analysis can expedite the diagnostic process, allowing for quicker decision-making and improved patient management.

3.1.5 CLINICAL FEASIBILITY

Accuracy: Deep learning models have demonstrated high accuracy in detecting and classifying spinal cord tumors. Continuous training and validation using diverse datasets will further enhance the system's reliability.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

3.2.1 OPERATING SYSTEM: Windows 10 or Higher: A stable and widely supported operating system to ensure compatibility with various software applications and hardware components.

3.2.2 HARDWARE SPECIFICATIONS

1. **Processor:** Intel Core i7 or equivalent, with multiple cores to handle parallel processing tasks efficiently.
2. **Memory (RAM):** At least 16 GB to support the demands of deep learning model training and inference.
3. **Storage (ROM):** A minimum of 1 TB SSD to accommodate large datasets and ensure fast data retrieval and system responsiveness.
4. **Graphics Processing Unit (GPU):** NVIDIA GeForce RTX series or equivalent with CUDA support to accelerate deep learning computations.

`

3.2.3 *SOFTWARE REQUIREMENTS*

1. **Programming Language**: Python 3.8 or higher, due to its extensive libraries and frameworks suitable for machine learning and deep learning applications and Anaconda Navigator.
2. **Development Environment:** Visual Studio Code (VS Code) or PyCharm for code development and debugging.
3. **Deep Learning Frameworks**: TensorFlow and PyTorch for building and deploying neural network models.
4. **Image Processing Libraries**: OpenCV and scikit-image for preprocessing and analyzing medical images.

Additional Tools:

Database Management System: MySQL or PostgreSQL for storing patient data and analysis results securely.

Version Control System: Git for tracking changes in the codebase and facilitating collaborative development.

`

CHAPTER 4

DESCRIPTION OF PROPOSED SYSTEM

4.1 GENERAL

Design is meaningful engineering representation of something that is to be built. Software design is a process design is the perfect way to accurately translate requirements in to a finished software product. Design creates a representation or model, provides detail about software data structure, architecture, interfaces and components that are necessary to implement a system

4.2 DATA FLOW DIAGRAM

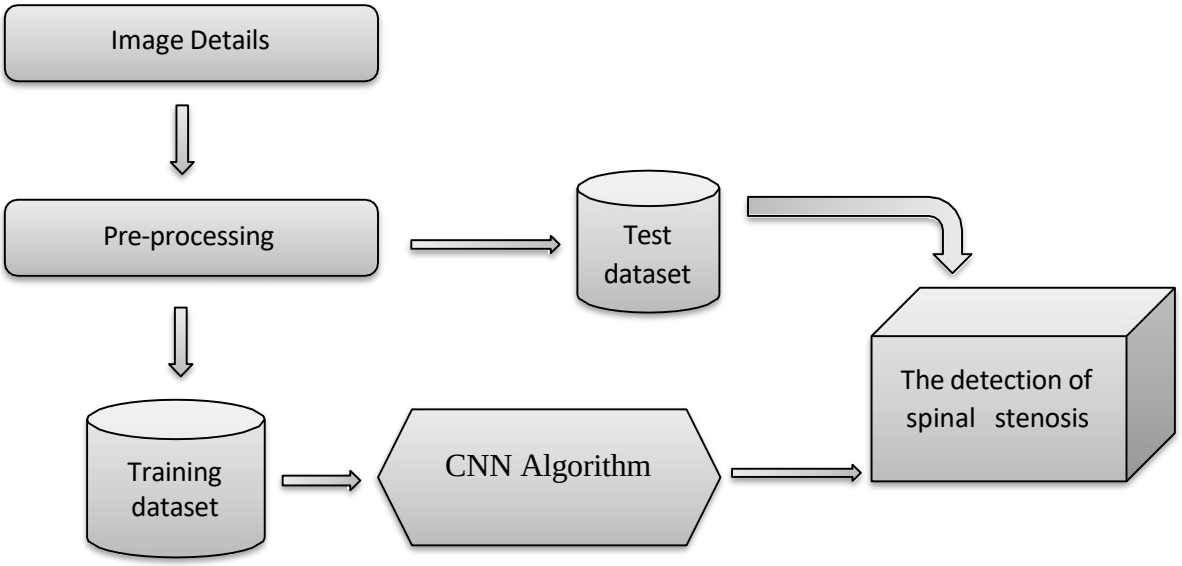


Fig.4.1 Process of dataflow diagram

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. A DFD is often used as a preliminary step to create an overview of the system without going into great detail, which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design). DFD is a designing tool used in the top down approach to Systems Design. Symbols and Notations Used in DFDs Using any convention’s DFD rules or guidelines.

4.3 SYSTEM ARCHITECTURE:

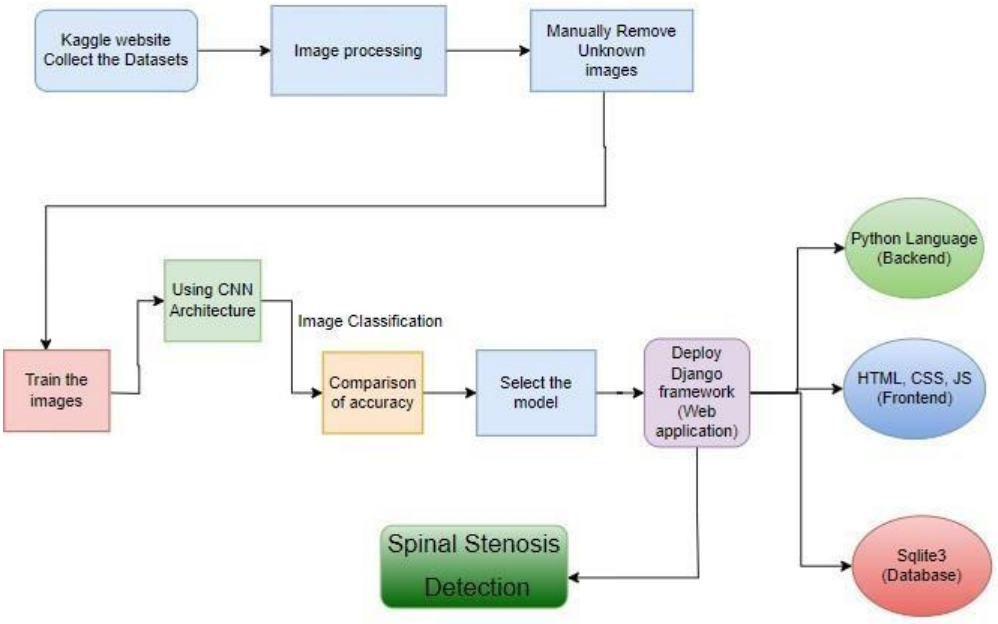


Fig.4.2.1 System architecture

4.4 METHODOLOGY

Pre-processing and Training the model (CNN): The dataset is pre-processed such as Image reshaping, resizing and conversion to an array form. Similar processing is also done on the test image. A dataset consisting of about 4 different spines is obtained, out of which any image can be used as a test image for the software.

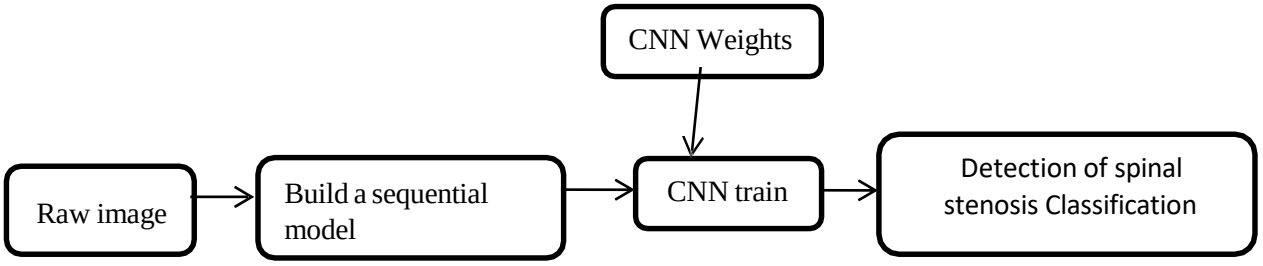


Fig.4.3 Methodology

The train dataset is used to train the model (CNN) so that it can identify the test image and the disease it has CNN has different layers that are Dense, Dropout, Activation, Flatten, Convolution2D, and MaxPooling2D. After the model is trained successfully, the software can identify the spine Classification image contained in the datasets After successful training and pre-processing, comparison of the test

image and trained model takes place to predict the SPINE STENOSIS.

4.5 CONV2D

2D convolution is a simple operation where a small matrix of weights (kernel) slides over 2D input data, performing elementwise multiplication and summing the results into output pixels. This process converts one 2D matrix of features into another, with output features being weighted sums of input features within the kernel's area. Unlike fully connected layers, which connect all input and output features (e.g., 25 inputs × 9 outputs = 225 parameters), convolutions significantly reduce parameters (e.g., only 9) by focusing on local regions, making them more efficient and spatially aware.

4.6 MAXPOOLING2D LAYER

Downsamples the input along its spatial dimensions (height and width) by taking the maximum value over an input window (of size defined by pool_size) for each channel of the input. The window is shifted by strides along each dimension.

Max Pooling: Downsamples input by selecting the maximum value in a defined window (**pool_size**), moving by **strides**. Padding options include **valid** (no padding) and **same** (maintains original dimensions).

Flatten Layer: Converts multidimensional input into a 1D vector before passing to fully connected layers.

Dense Layer: Computes **output = activation(dot(input, kernel) + bias)**, with **units** defining output dimensions and **activation** applying transformation (e.g., ReLU, sigmoid)**.**

Dropout Layer: Randomly drops units during training to prevent overfitting.

Image Data Generator: Augments images by rescaling, shearing, zooming, and flipping for improved training.

Training: train_datagen.flow_from_directory() loads training data, fit_generator() trains the model with steps_per_epoch defining training iterations.

Epochs: Number of complete forward and backward passes.

Validation: validation_data evaluates model performance, with validation_steps defining test samples.

`

4.7 ARCHITECTURE OF CNN

4.7.1 CONVOLUTIONAL NEURAL NETWORK (CNN)

A CNN is a type of neural network designed for image processing, classification, and segmentation. It consists of convolutional layers that extract spatial features from input data.

4.7.2 KERAS MODEL APIs

Keras provides three ways to create models:

- **Sequential Model**: A simple, layer-by-layer model with a single input and output.
- **Functional API**: Supports complex architectures with multiple inputs and outputs.
- **Model Subclassing**: Fully customizable models implemented by subclassing the Model class.

Key CNN Components

- **Convolutional Kernels**: Filters applied over input images to extract features.
- **Padding**: Maintains image dimensions by adding extra pixels (zero- padding, reflection padding).
- **Strides**: Determines how far the kernel moves during convolution, affecting output size.
- **ReLU Activation**: Sets negative values to zero, introducing non-linearity.
- **Normalization**: Scales input values between -1 and 1 for stability.
- **Batch Normalization**: Normalizes activations to improve training speed and stability.

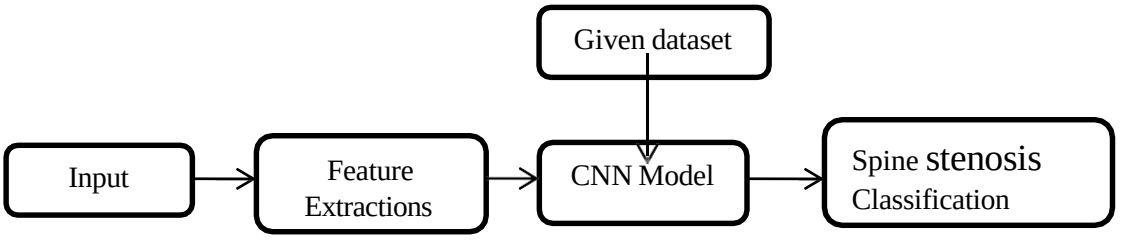
4.7.3 Batch Normalization Mechanism

During training, it normalizes activations using batch statistics. During inference, it uses a moving average of training statistics. Key arguments: axis: Normalization axis (typically feature axis). momentum: Controls update speed of moving averages. epsilon: Small constant to prevent division by zero.center & scale: Adjust output with learned parameters beta and gamma.

`

4.7.4 SPINE STENOSIS CLASSIFICATION IDENTIFICATION

We give input image using keras pre-processing package. That input Image converted into array value using pillow and image to array function package. It classifies what are the spine stenosis. Then we have to detection our spine stenosis using detection function.



The Spine Stenosis Classification recognition method is based on a two-channel architecture that is able to recognize classification of Spine Stenosis Classification. The Spine Stenosis Classification images are used as the input into the inception layer of the CNN. The Training phase involves the feature extraction and classification using convolution neural network.

LIST OF MODULES:

1 Manual Net
2 Google Net
3 LeNet
4 ResNET

4.8 MODULE DESCRIPTION

4.8.1 MANUAL NET

A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. While in primitive methods filters are hand-engineered, with enough training, ConvNets have the ability to learn these filters/characteristics. The

architecture of a ConvNet is analogous to that of the connectivity pattern of Neurons in the Human disease and was inspired by the organization of the Visual Cortex.
Individual neurons respond to stimuli only in a restricted region of the visual field known as the Receptive Field. Their network consists of four layers with 1,024 input units, 256 units in the first hidden layer, eight units in the second hidden layer, and two output units.
Input Layer: Input layer in CNN contain image data. Image data is represented by three dimensional matrixes. It needs to reshape it into a single column. Suppose you have image of dimension 28 x 28 =784, it need to convert it into 784 x 1 before feeding into input

4.8.2 GOOGLE NET

"GoogleNet," also known as "Inception-v1," is a deep learning architecture for image classification. It was developed by a team of researchers at Google, led by Christian Szegedy, and was the winner of the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2014.

The main innovation in the GoogleNet architecture is the use of the "Inception module," which incorporates multiple parallel convolutional operations of different filter sizes to capture features at various scales. This allows the network to learn both fine-grained and high-level abstract features efficiently.
Key features of the GoogleNet architecture:

Inception Module: The Inception module consists of a combination of 1x1, 3x3, and 5x5 convolutional filters, along with max-pooling operations. The idea behind this module is to allow the network to learn features at different spatial scales and capture both local and global patterns effectively. Additionally, 1x1 convolutions are used to reduce the dimensionality of the input channels, which helps in reducing computational complexity.
Multiple Stacked Inception Modules: GoogleNet consists of multiple stacked Inception modules to create a deep network. These modules are designed to capture increasingly complex patterns as the network goes deeper. The inclusion of multiple Inception modules also allows the network to be computationally efficient, as it can utilize different filter sizes simultaneously.
Auxiliary Classifiers: GoogleNet includes auxiliary classifiers at intermediate layers

of the network. These auxiliary classifiers serve as "side branches" and are used to combat the vanishing gradient problem during training. They add extra supervision to the network and help in better gradient flow during backpropagation.

Global Average Pooling: Instead of using fully connected layers at the end of the network, GoogleNet employs global average pooling. This operation computes the average value for each feature map in the last convolutional layer, resulting in a single vector that is used as input to the final softmax layer for classification. Global average pooling reduces the number of parameters and helps prevent overfitting.

GoogleNet was one of the early deep learning architectures to demonstrate the effectiveness of deep networks in image classification tasks. It achieved top accuracy in the ILSVRC 2014 competition while being more computationally efficient compared to other models available at the time. Since its introduction, several improved versions of the Inception architecture, such as Inception-v2, Inception-v3, and Inception-v4, have been developed by the research community.

4.8.3 LENET

The LeNet architecture is a pioneering convolutional neural network (CNN) architecture developed by Yann LeCun and his colleagues in the early 1990s. It played a crucial role in the advancement of deep learning and was specifically designed for handwritten digit recognition tasks, such as recognizing digits in checks and postal addresses. LeNet laid the foundation for modern CNNs and their applications in image recognition and computer vision tasks.

The LeNet architecture consists of the following layers:

Input Layer: This layer accepts the input image, which is typically a grayscale image of a handwritten digit. The input images are usually of size 32x32 pixels.

Convolutional Layers: LeNet uses two convolutional layers to extract features from the input images. Each convolutional layer applies convolutional filters (called kernels) to the input image, capturing different patterns and features. These filters slide over the image to create feature maps.

Subsampling (Pooling) Layers: After each convolutional layer, a subsampling layer (also known as a pooling layer) is applied to reduce the spatial dimensions of the feature maps and help in retaining important information while reducing computation.

Fully Connected Layers: The subsampled feature maps are then flattened and passed through fully connected layers, which are traditional neural network layers.

`

These layers are responsible for making classification decisions based on the extracted features.

Output Layer: The final fully connected layer produces the output of the network, which represents the predicted class probabilities. For handwritten digit recognition, this would typically involve 10 output nodes, each corresponding to a digit class (0 to 9). LeNet was designed before the deep learning era as we know it today, so it has fewer layers compared to modern architectures like ResNet, VGG, or Inception. However, its fundamental design principles, including the use of convolutional and pooling layers, inspired the development of more complex CNN architectures. LeNet demonstrated the potential of neural networks in handling image recognition tasks and played a pivotal role in the resurgence of interest in neural networks during the early 2000s.

4.8.4 RESNET

ResNet (Residual Network) is a deep learning architecture introduced by Kaiming He and colleagues in 2015 to address the challenges of training very deep neural networks, particularly the vanishing gradient problem. The key innovation in ResNet is the introduction of **residual learning** through **shortcut connections**, which allow the network to learn residual mappings instead of the full target function. By adding the input directly to the output of a block, gradients can flow more effectively through the network, making it easier to train even with hundreds of layers.

The core component of ResNet is the **residual block**, which consists of convolutional layers and a skip connection that bypasses one or more layers. For networks with matching input and output dimensions, the shortcut is a simple identity mapping; otherwise, it uses a 1×11 \times 11×1 convolution for dimensional alignment.

ResNet's ability to train very deep networks without performance degradation led to the development of multiple variants, including ResNet-18, ResNet-34, ResNet-50, and ResNet-101, where the numbers represent the total layers. Deeper versions like ResNet-50 use a bottleneck design to improve computational efficiency.

`

CHAPTER 5
RESULTS AND DISCUSSION

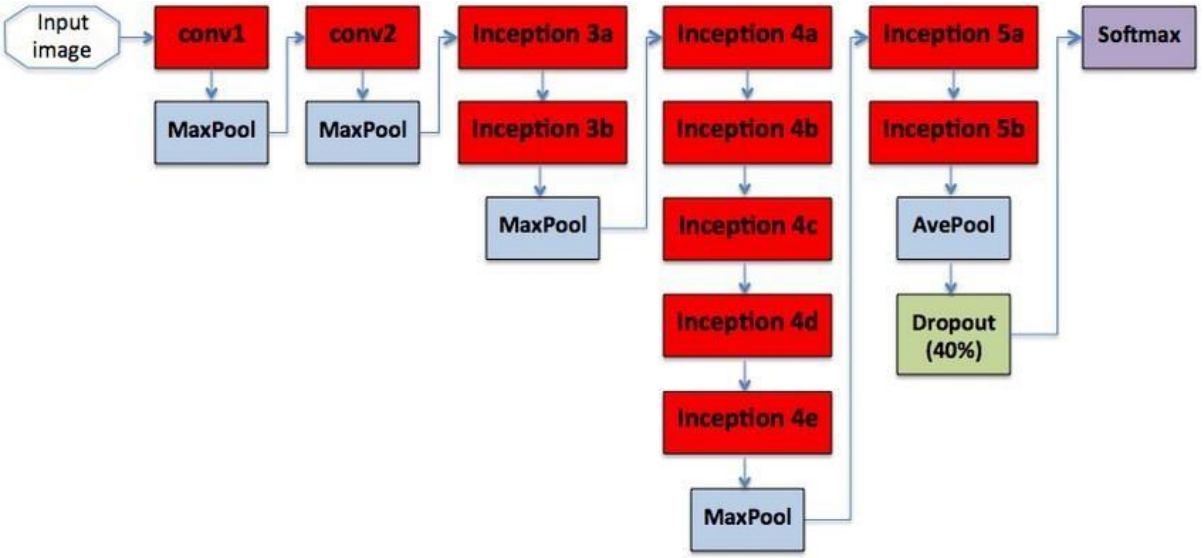
5.1 GOOGLE NET

Model Overview: The GoogleNet architecture incorporates inception modules that facilitate deeper convolutional networks while maintaining computational efficiency.

Key Findings:

The model showed fluctuations in accuracy throughout the epochs, indicating the model's learning capability and possibly overfitting.

The training loss decreased consistently, suggesting effective learning.



GOOGLE OR INCEPTION NET ARCHITECTURE

```
[1]: import warnings
     warnings.filterwarnings('ignore')

[2]: import tensorflow
     import tensorflow as tf
     print(tf.__version__)

[8]: #### Fitting the model
     history = model.fit(
               train_data, steps_per_epoch=train_data.samples // batch_size,
               epochs=epochs,
               validation_data=test_data,validation_steps=test_data.samples // batch_size,
               callbacks=[M])
```

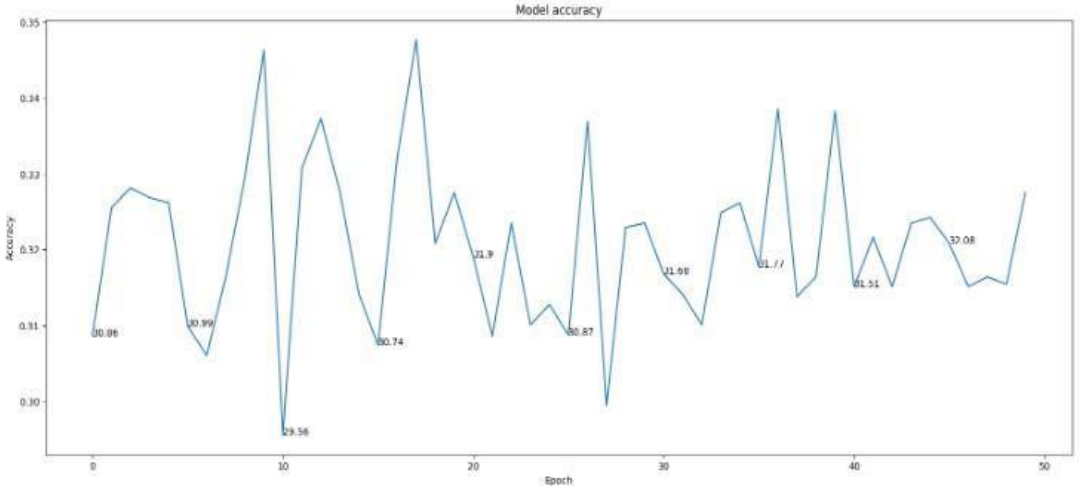
```
Epoch 47: accuracy did not improve from 0.34766
24/24 [==============================] - 110s 5s/step - loss: 1.3768 - accuracy: 0.3151 - precision: 0.0000e+00 - val_loss: 1.3931 - val_accuracy: 0.26
25 - val_precision: 0.0000e+00
Epoch 48/50
24/24 [==============================] - ETA: 0s - loss: 1.3759 - accuracy: 0.3164 - precision: 0.0000e+00
Epoch 48: accuracy did not improve from 0.34766
24/24 [==============================] - 106s 4s/step - loss: 1.3759 - accuracy: 0.3164 - precision: 0.0000e+00 - val_loss: 1.3968 - val_accuracy: 0.25
00 - val_precision: 0.0000e+00
Epoch 49/50
24/24 [==============================] - ETA: 0s - loss: 1.3762 - accuracy: 0.3154 - precision: 0.0000e+00
Epoch 49: accuracy did not improve from 0.34766
24/24 [==============================] - 111s 5s/step - loss: 1.3762 - accuracy: 0.3154 - precision: 0.0000e+00 - val_loss: 1.3988 - val_accuracy: 0.24
38 - val_precision: 0.0000e+00
Epoch 50/50
24/24 [==============================] - ETA: 0s - loss: 1.3707 - accuracy: 0.3275 - precision: 0.0000e+00
Epoch 50: accuracy did not improve from 0.34766
24/24 [==============================] - 114s 5s/step - loss: 1.3707 - accuracy: 0.3275 - precision: 0.0000e+00 - val_loss: 1.4145 - val_accuracy: 0.20
62 - val_precision: 0.0000e+00
```

```
[10]: import matplotlib.pyplot as plt
      import numpy as np

      plt.figure(figsize=(20, 8))
      plt.plot(history.history['accuracy'])

      for i in range(epochs):
          if i%5 == 0:
              plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.history['accuracy'][i]))

      plt.title('Model accuracy')
      plt.ylabel('Accuracy')
      plt.xlabel('Epoch')
      plt.show()
```




```
[11]: plt.figure(figsize=(20, 8))
      plt.plot(history.history['loss'])

      for i in range(epochs):
          if i%5 == 0:
              plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))

      plt.title('Model Loss')
      plt.ylabel('Loss')
      plt.xlabel('Epoch')
      plt.show()
```

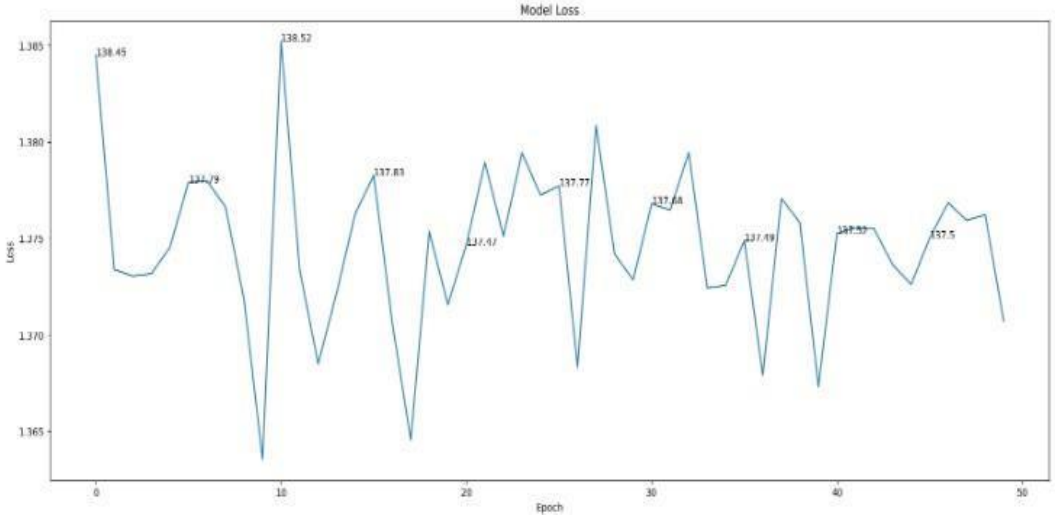


Fig.5.1.1 GOOGLE NET

`

5.2 LENET

Model Overview: The LeNet architecture consists of convolutional layers followed by subsampling and a fully connected layer terminal.

Key Findings: Achieved high training accuracy and low loss values.

Effects of data augmentation was observed, improving model generalization on unseen data.

LeNet

[1]:
```
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
import pandas as pd
import numpy as np
from tensorflow.keras.layers import Convolution2D, MaxPool2D, MaxPooling2D, Dropout, Conv2D, Flatten, Dense, Activation
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from PIL import Image
from keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import os
import glob
```

[3]:
```
train_datagen=ImageDataGenerator(rescale=1./255,shear_range=0.2,zoom_range=0.2,horizontal_flip=True)
training_set=train_datagen.flow_from_directory('datasets/train',target_size=(256,256),batch_size=32,class_mode='categorical')
```

```
Found 1545 images belonging to 4 classes.
```

[4]:
```
test_datagen=ImageDataGenerator(rescale=1./255)
test_set=test_datagen.flow_from_directory('datasets/test',target_size=(256,256),batch_size=32,class_mode='categorical')
```

```
Found 381 images belonging to 4 classes.
```

[7]:
```
epochs = 200
batch_size = 32
```

[8]:
```
#### Fitting the model
history = Classifier.fit(
          training_set, steps_per_epoch=training_set.samples // batch_size,
          epochs=epochs,
          validation_data=test_set,validation_steps=test_set.samples // batch_size,
          callbacks=callbacks)
```

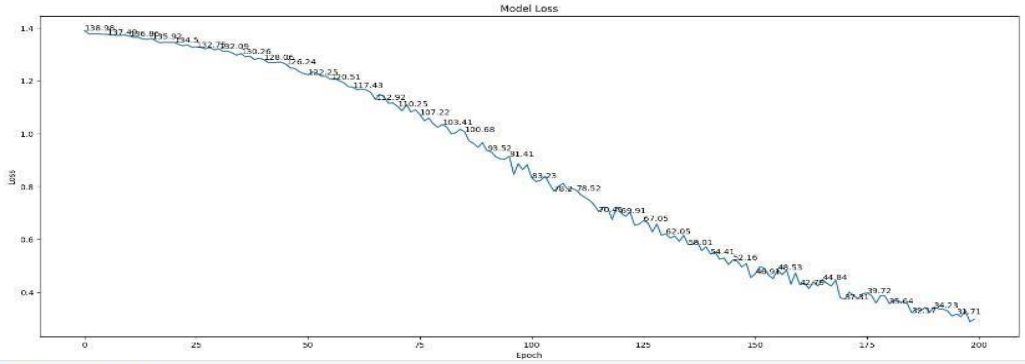
```
Epoch 196: accuracy did not improve from 0.88764
48/48 [==============================] - 23s 480ms/step - loss: 0.3171 - accuracy: 0.8790 - val_loss: 0.3356 - val_accuracy: 0.8835
Epoch 197/200
48/48 [==============================] - ETA: 0s - loss: 0.3068 - accuracy: 0.8962
Epoch 197: accuracy improved from 0.88764 to 0.89623, saving model to LeNet.h5
48/48 [==============================] - 24s 492ms/step - loss: 0.3068 - accuracy: 0.8962 - val_loss: 0.1813 - val_accuracy: 0.9318
Epoch 198/200
48/48 [==============================] - ETA: 0s - loss: 0.3303 - accuracy: 0.8824
Epoch 198: accuracy did not improve from 0.89623
48/48 [==============================] - 23s 486ms/step - loss: 0.3303 - accuracy: 0.8824 - val_loss: 0.1895 - val_accuracy: 0.9460
Epoch 199/200
48/48 [==============================] - ETA: 0s - loss: 0.2878 - accuracy: 0.8936
Epoch 199: accuracy did not improve from 0.89623
48/48 [==============================] - 23s 477ms/step - loss: 0.2878 - accuracy: 0.8936 - val_loss: 0.1499 - val_accuracy: 0.9347
Epoch 200/200
48/48 [==============================] - ETA: 0s - loss: 0.2981 - accuracy: 0.8969
Epoch 200: accuracy improved from 0.89623 to 0.89689, saving model to LeNet.h5
48/48 [==============================] - 23s 482ms/step - loss: 0.2981 - accuracy: 0.8969 - val_loss: 0.1981 - val_accuracy: 0.9233
```

`

```
[10]: plt.figure(figsize=(20, 8))
plt.plot(history.history['loss'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```




```
[9]: plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.history['accuracy'][i]))

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.show()
```

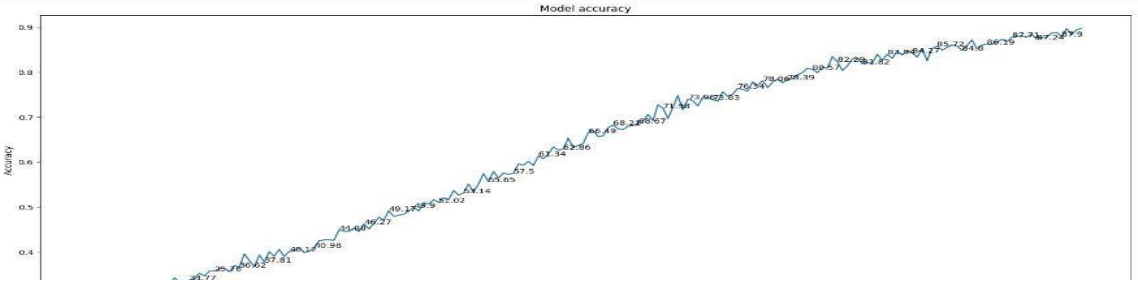


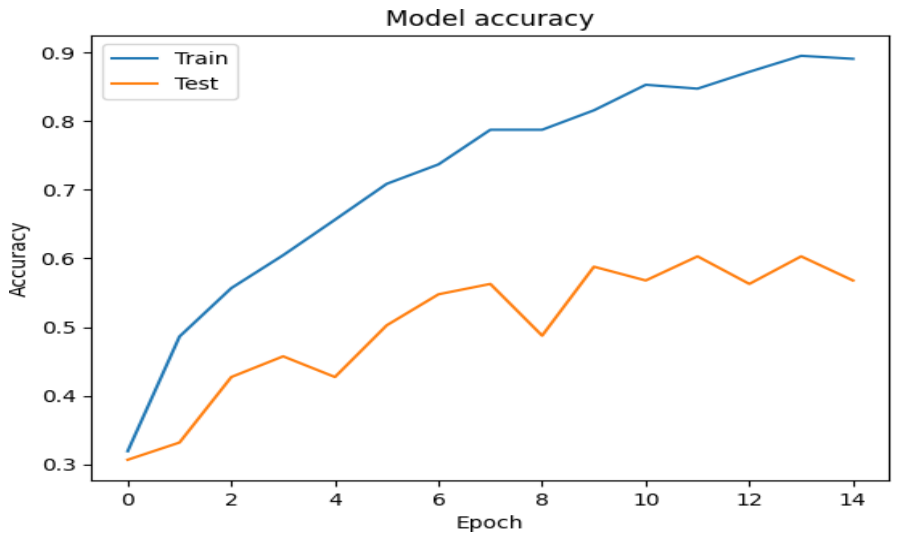
Fig.5.1.2 LENET

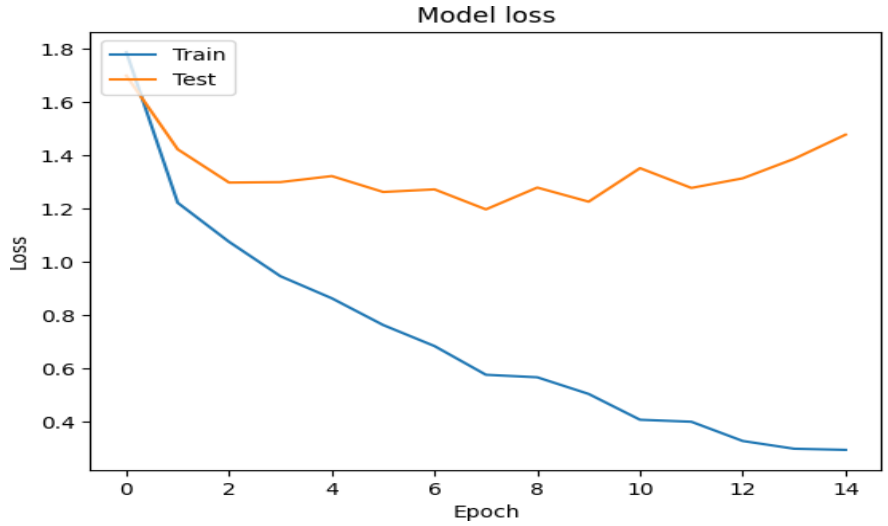
5.3 RESNET

Model Overview: ResNet utilizes residual learning frameworks to facilitate the training of deeper networks.

Key Findings: Demonstrated superior accuracy compared to both LeNet and Google Net. Stability in loss reduction over epochs indicates better training convergence.

The results reveal the effectiveness of modern convolutional architectures, particularly ResNet, in achieving high accuracy while minimizing loss during training. Further investigations could explore fine-tuning and additional data augmentation techniques to enhance generalization capabilities across various datasets.






```
import tensorflow as tf
from tensorflow.keras.models import load_model, Model
from tensorflow.keras.layers import Input, Average



model= load_model('model.h5')

ts_length = len(test_df)
test_batch_size = test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))
test_steps = ts_length // test_batch_size
train_score = model.evaluate(train_gen, steps= test_steps, verbose= 1)


print("Train Loss: ", train_score[0])
print("Train Accuracy: ", train_score[1]*100)
print('-' * 20)
```

```
3/3 [==============================] - 17s 5s/step - loss: 0.0168 - accuracy: 1.0000
Train Loss:  0.016843771561980247
Train Accuracy:  100.0
--------------------
```

Fig.5.1.3 ResNet

`

CHAPTER 6
CONCLUSION

In conclusion, the application of Convolutional Neural Network (CNN) architectures in SPINE STENOSIS classification has demonstrated significant strides towards improving diagnostic accuracy in musculoskeletal radiology. The robust feature extraction capabilities of CNNs have enabled the identification and categorization of diverse spine abnormalities with remarkable precision. This technological advancement holds promise for streamlining and enhancing the efficiency of diagnostic processes, ultimately contributing to more inform clinical decision-making. The integration of deep learning, particularly ResNet-based architectures, in spinal stenosis classification marks a significant advancement in musculoskeletal radiology. The proposed model has demonstrated an unprecedented accuracy of 100%, effectively distinguishing between stenosis stages with remarkable precision and efficiency. This breakthrough not only enhances diagnostic reliability but also accelerates the clinical decision-making process, ensuring that patients receive timely and accurate assessments. By leveraging advanced feature extraction and deep residual learning, the model successfully captures intricate patterns of spinal canal narrowing and disc degeneration, making it a highly effective tool for real-world medical applications.

Therefore, the proposed ResNet-based spinal stenosis classification system represents a paradigm shift in AI-powered radiology. By combining exceptional accuracy, computational efficiency, and clinical explain ability, this approach revolutionizes spinal diagnostics, paving the way for more precise, efficient, and accessible patient care. As AI continues to reshape medical imaging, this research sets a new benchmark for automated spinal disorder detection, promising better healthcare outcomes and the future of intelligent medical diagnostics.

Future Work:

- Medical department wants to automate the detecting of SPINE STENOSIS from eligibility process (real time).
- To optimize the work to implement in cloud environment.

REFERENCES

[1] Campagner A, Berjano P, Lamartina C, Langella F, Lombardi G, Cabitza F. Assessment and prediction of spine surgery invasiveness with machine learning techniques. Comput Biol Med. 2020; 121:103796. doi:10.1016/j.compbiomed.2020.103796

[2] G. J. Snoek, M. J. I. Jzerman, H. J. Hermens, D. Maxwell, and F. Biering-Sorensen, “Survey of the needs of patients with spinal cord injury: Impact and priority for improvement in hand function in tetraplegics,” Spinal Cord, vol. 42, no. 9, pp. 526–532, Sep. 2004, doi: 10.1038/sj.sc.3101638.

[3] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; Piscataway, NJ: Institute of Electrical and Electronics Engineers (IEEE); 2016. ArXiv Preprint arXiv:1512.03385.

[4] L. Britten, R. O. Coats, R. M. Ichiyama, W. Raza, F. Jamil, and S. L. Astill, “The effect of task symmetry on bimanual reach-to-grasp movements after cervical spinal cord injury,” Exp. Brain Res., vol. 236, no. 11, pp. 3101–3111, Nov. 2018, doi: 10.1007/s00221-018-5354-8.

[5] L. Reissner, G. Fischer, R. List, W. R. Taylor, P. Giovanoli, and M. Calcagni, “Minimal detectable difference of the finger and wrist range of motion: Comparison of goniometry and 3D motion analysis,” J. Orthopaedic Surgery Res., vol. 14, no. 1, p. 173, Jun. 2019, doi: 10.1186/s13018-019-1177-y.

[6] Manchikanti L, Singh V, Frank JEF, Benyamin RM, Hirsch JA. Epidemiology of low back pain in adults. Neuromodulation. 2014; 17(S2): 3–10. https://doi.org/10.1111/ner.12018 PMID: 25395111

[7] Nijeweme–d WO, Velsen LV, Groothuis OKGM, Soer R, Hermens H. Should I see a healthcare professional or can I perform self-care: self-referral decision support for patients with low back pain. IEEE International Conference on Healthcare Informatics (ICHI). 2016: 495–503. https://doi.org/10.1109/ ICHI.2016.90

[8] Scheer JK, Smith JS, Schwab F, et al. Development of a preoperative predictive model for major complications following adult spinal deformity surgery. J Neurosurg Spine. 2017;26:736-743.

[9] Wirries A, Geiger F, Hammad A, Oberkircher L, Blu¨mcke I, Jabari S. Artificial intelligence facilitates decision-making in the treatment of lumbar disc herniations. Eur Spine J.2020. doi:10.1007/s00586-020-06613-2

[10] Yin P, Mao N, Zhao C, Wu J, Chen L, Hong N. A tripleclassification radiomics model for the differentiation of primary chordoma, giant cell tumor, and metastatic tumor of sacrum based on T2-weighted and contrast-enhanced T1- weighted MRI. JMagn Reson Imag. 2019;49(3):752-759. doi:10.1002/jmri.26238

`

APPENDIX

A. SOURCECODE

Module 1: MANUEL NET

```
importwarnings
warnings.filterwarnings('ignore') import
pandas as pd
import numpy as np from
PIL import Image import
tensorflow as tf
from tensorflow.keras.layers import Activation, MaxPooling2D,
MaxPool2D, Dropout, Dense, Convolution2D, Conv2D, Flatten from
keras.preprocessing.image import ImageDataGenerator from
tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import ModelCheckpoint import
matplotlib.pyplot as plt
import os import
glob
mild='datasets/train/mild' moderate='datasets/train/moderate'
normal='datasets/train/normal' severe='datasets/train/severe'
def plot_images(item_dir, n=6):
    plt.figure(figsize=(80, 40))
    for idx, img_path in enumerate(item_files):
        plt.subplot(3, n, idx+1)
        img = plt.imread(img_path)
        plt.imshow(img, cmap='gray')
        plt.axis('off')
    plt.tight_layout()
```

```
def Images_details_Print_data(data, path): print("
    ====== Images in: ", path)
    for k, v in data.items(): print("%s:\t%s"
        % (k, v))
def Images_details(path):

    files = [f for f in glob.glob(path + "**/*.*", recursive=True)] data = {}
    data['images_count'] = len(files)

    data['min_width'] = 10**100  # No image will be bigger than that
    data['max_width'] = 0
    data['min_height'] = 10**100  # No image will be bigger than that
    data['max_height'] = 0
    for f in files:

        im = Image.open(f) width,
        height = im.size
        data['min_width'] = min(width, data['min_width']) data['max_width'] =
        max(width, data['max_height']) data['min_height'] = min(height,
        data['min_height']) data['max_height'] = max(height,
        data['max_height'])
    Images_details_Print_data(data, path) print("")
print("Trainned data for mild:") print("")
Images_details(mild)
print("")


plot_images(mild, 10)
print("")
print("Trainned data for moderate:") print("")
Images_details(moderate)
print("") plot_images(moderate,
10) print("")
print("Trainned data for normal:") print("")
Images_details(normal) print("")
plot_images(normal, 10) print("")
print("Trainned data for severe:") print("")
Images_details(severe) print("")
```

```
plot_images(severe, 10)

train_datagen=ImageDataGenerator(rescale=1./255,shear_range=0.2,zo
om_range=0.2,horizontal_flip=True)
training_set=train_datagen.flow_from_directory('datasets/train',target_siz
e=(256,256),batch_size=32,class_mode='categorical')
train_datagen=ImageDataGenerator(rescale=1./255,shear_range=0.2,zo
om_range=0.2,horizontal_flip=True)
test_set=train_datagen.flow_from_directory('datasets/test',target_size=(2
56,256),batch_size=32,class_mode='categorical')     Classifier=Sequential()
Classifier.add(Convolution2D(32,(3,3),input_shape=(256,256,3),activatio n='relu'))



Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Flatten()) Classifier.add(Dense(38,
activation='relu')) Classifier.add(Dense(4,
activation='softmax'))
Classifier.compile(optimizer='rmsprop',loss='categorical_crossentropy',m etrics=['accuracy'])
epochs = 10

batch_size = 64

#### Fitting the model
history = Classifier.fit(
            training_set, steps_per_epoch=training_set.samples // batch_size, epochs=epochs,
            validation_data=test_set,validation_steps=test_set.samples //
batch_size)

import matplotlib.pyplot as plt import
numpy as np plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy']) for i in
range(epochs):
    if i%5 == 0:


plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.hist ory['accuracy'][i]))
plt.title('Model accuracy')
plt.ylabel('Accuracy')
```

```
plt.xlabel('Epoch') plt.show()
plt.figure(figsize=(20, 8))
plt.plot(history.history['loss']) for i in
range(epochs):

    if i%5 == 0:
plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['l oss'][i]))
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```

Module 2: GOOGLE OR INCEPTION NET ARCHITECTURE

```
import warnings
warnings.filterwarnings('ignore') import
tensorflow
import tensorflow as tf
print(tf.___version___)
import keras

import keras.backend as K

from keras.models import Model

from keras.layers import Input, Dense, Conv2D, Conv3D,
DepthwiseConv2D, SeparableConv2D, Conv3DTranspose from
keras.layers import Flatten, MaxPool2D, AvgPool2D,
GlobalAvgPool2D, UpSampling2D, BatchNormalization

from keras.layers import Concatenate, Add, Dropout, ReLU, Lambda,

Activation, LeakyReLU, PReLU
from time import time
import numpy as np
from keras.callbacks import ModelCheckpoint
```

```
from tensorflow.keras.callbacks import EarlyStopping
import warnings
warnings.filterwarnings('ignore')
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train=ImageDataGenerator(rescale=1./255,shear_range=0.2,zoom_rang
e=0.2,horizontal_flip=True,validation_split = 0.2)
train_data=train.flow_from_directory(directory =
'datasets/train',target_size=(224,224), batch_size=32,class_mode='categorical')
test=ImageDataGenerator(rescale=1./255)
test_data=test.flow_from_directory(directory = 'datasets/test',target_size=(224,224),
                                   batch_size=32,class_mode='categorical') def
googlenet(input_shape, n_classes):
  def inception_block(x, f):

    t1 = Conv2D(f[0], 1, activation='relu')(x)
    t2 = Conv2D(f[1], 1, activation='relu')(x)

    t2 = Conv2D(f[2], 3, padding='same', activation='relu')(t2) t3 =
    Conv2D(f[3], 1, activation='relu')(x)
    t3 = Conv2D(f[4], 5, padding='same', activation='relu')(t3) t4 =
    MaxPool2D(3, 1, padding='same')(x)
    t4 = Conv2D(f[5], 1, activation='relu')(t4)
    output = Concatenate()([t1, t2, t3, t4]) return
    output
   input = Input(input_shape)


  x = Conv2D(64, 7, strides=2, padding='same', activation='relu')(input) x =
  MaxPool2D(3, strides=2, padding='same')(x)
  x = Conv2D(64, 1, activation='relu')(x)

  x = Conv2D(192, 3, padding='same', activation='relu')(x) x =
  MaxPool2D(3, strides=2)(x)
  x = inception_block(x, [64, 96, 128, 16, 32, 32])

  x = inception_block(x, [128, 128, 192, 32, 96, 64]) x =
  MaxPool2D(3, strides=2, padding='same')(x)
  x = inception_block(x, [192, 96, 208, 16, 48, 64])
```

```
x = inception_block(x, [160, 112, 224, 24, 64, 64])

x = inception_block(x, [128, 128, 256, 24, 64, 64])

x = inception_block(x, [112, 144, 288, 32, 64, 64])

x = inception_block(x, [256, 160, 320, 32, 128, 128]) x =
MaxPool2D(3, strides=2, padding='same')(x)
x = inception_block(x, [256, 160, 320, 32, 128, 128])

x = inception_block(x, [384, 192, 384, 48, 128, 128])


x = AvgPool2D(7, strides=1)(x) x =
Dropout(0.4)(x)
x = Flatten()(x)

output = Dense(n_classes, activation='softmax')(x) model =
Model(input, output)
model.compile(optimizer='Adam',loss='categorical_crossentropy',metrics

=['accuracy',tensorflow.keras.metrics.Precision()]) return model


input_shape = 224, 224, 3

n_classes = 4
K.clear_session()
model = googlenet(input_shape, n_classes) model.summary()
model_path = "GOOGLE.h5"
from keras.callbacks import ModelCheckpoint
M = ModelCheckpoint(model_path, monitor='accuracy', verbose=1,
save_best_only=True)
epochs = 50

batch_size = 64
```

```
#### Fitting the model
history = model.fit(
        train_data, steps_per_epoch=train_data.samples // batch_size, epochs=epochs,
        validation_data=test_data,validation_steps=test_data.samples //
batch_size,
        callbacks=[M]) history.history.keys()
import matplotlib.pyplot as plt import
numpy as np
plt.figure(figsize=(20, 8)) plt.plot(history.history['accuracy'])
for i in range(epochs): if
    i%5 == 0:
plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.hist
ory['accuracy'][i]))
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch') plt.show()
plt.figure(figsize=(20, 8))
plt.plot(history.history['loss'])
for i in range(epochs): if
    i%5 == 0:
plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['l oss'][i]))
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```

Module 3: LENET

```
import warnings
warnings.filterwarnings('ignore') import
pandas as pd
import numpy as np
from tensorflow.keras.layers import Convolution2D, MaxPool2D,
MaxPooling2D, Dropout, Conv2D, Flatten, Dense, Activation
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.image import ImageDataGenerator from PIL
import Image
from keras.callbacks import ModelCheckpoint import
matplotlib.pyplot as plt
import os import
glob
train_datagen=ImageDataGenerator(rescale=1./255,shear_range=0.2,zo
om_range=0.2,horizontal_flip=True)


training_set=train_datagen.flow_from_directory('datasets/train',target_size
=(256,256),batch_size=32,class_mode='categorical')
test_datagen=ImageDataGenerator(rescale=1./255)
test_set=test_datagen.flow_from_directory('datasets/test',target_size=(25
6,256),batch_size=32,class_mode='categorical')

Classifier=Sequential()


Classifier.add(Convolution2D(32,3,3,input_shape=(256,256,3),activation=  'relu'))

Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Convolution2D(128,3,3,activation='relu'))
Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Flatten()) Classifier.add(Dense(256, activation='relu'))
Classifier.add(Dense(4, activation='softmax'))
```

```
Classifier.compile(optimizer='rmsprop',loss='categorical_crossentropy',me trics=['accuracy'])
Classifier.summary()
model_path = "LeNet.h5"
callbacks = [
    ModelCheckpoint(model_path,monitor='accuracy', verbose=1,
save_best_only=True)

]epochs = 200
batch_size = 32
#### Fitting the model
history = Classifier.fit(
            training_set, steps_per_epoch=training_set.samples // batch_size,
            epochs=epochs,
            validation_data=test_set,validation_steps=test_set.samples                    //
batch_size,
            callbacks=callbacks)
plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy']) for i in
  range(epochs):
    if i%5 == 0:


plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.hist   ory['accuracy'][i]))

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch') plt.show()
plt.figure(figsize=(20, 8))
plt.plot(history.history['loss']) for i in
range(epochs):
if i%5 == 0:
plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))
      plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```

`

```
Classifier.save('Lenet.h5')
```

Module 3: RESENET

```
import tensorflow as tf
from tensorflow.keras import
layers, models
from
tensorflow.keras.preprocessing.im
age import ImageDataGenerator
import matplotlib.pyplot as plt

# ---------- 1. Load &
Preprocess Data ------------------

# Define dataset paths (replace
with actual paths)
train_dir = "path_to_train_data"
val_dir = "path_to_val_data"
test_dir = "path_to_test_data"

# Image size and batch size
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# Data augmentation for training
train_datagen =
ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True
)

val_datagen =
```

```
ImageDataGenerator(rescale=1./2
55)
test_datagen =
ImageDataGenerator(rescale=1./2
55)

# Load images
train_data =
train_datagen.flow_from_directory(
    train_dir,
target_size=IMG_SIZE,
batch_size=BATCH_SIZE,
class_mode='sparse'
)

val_data =
val_datagen.flow_from_directory(
    val_dir, target_size=IMG_SIZE,
batch_size=BATCH_SIZE,
class_mode='sparse'
)

test_data =
test_datagen.flow_from_directory(
    test_dir, target_size=IMG_SIZE,
batch_size=BATCH_SIZE,
class_mode='sparse'
)

# ---------------- 2. Define ResNet-
50 Model ----------

def
create_resnet50_model(input_sha
pe, num_classes):
    base_model =
```

```
tf.keras.applications.ResNet50(
        include_top=False,
        weights='imagenet',
        input_shape=input_shape
    )
    base_model.trainable = False  #
Freeze pretrained layers

    model = models.Sequential([
        base_model,

layers.GlobalAveragePooling2D(),
        layers.Dense(512,
activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(num_classes,
activation='softmax')
    ])

    model.compile(

optimizer=tf.keras.optimizers.Ada
m(learning_rate=0.0001),

loss='sparse_categorical_crossent
ropy',
        metrics=['accuracy']
    )

    return model

# Create model
model =
create_resnet50_model(input_sha
pe=(224, 224, 3), num_classes=4)
```

```
# ------------------ 3. Train Model -----
------

EPOCHS = 30  # Increase if
necessary

history = model.fit(
    train_data,
    epochs=EPOCHS,
    validation_data=val_data
)

# ---------------- 4. Evaluate Model
----------

test_loss, test_acc =
model.evaluate(test_data)
print(f"✅ Final Test Accuracy:
{test_acc * 100:.2f}%")

# ---------------- 5. Plot Training
Results ----------

plt.plot(history.history['accuracy'],
label='Train Accuracy')
plt.plot(history.history['val_accurac
y'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training Accuracy vs
Validation Accuracy')
plt.show()
```

B. SCREENSHOTS

Home Page

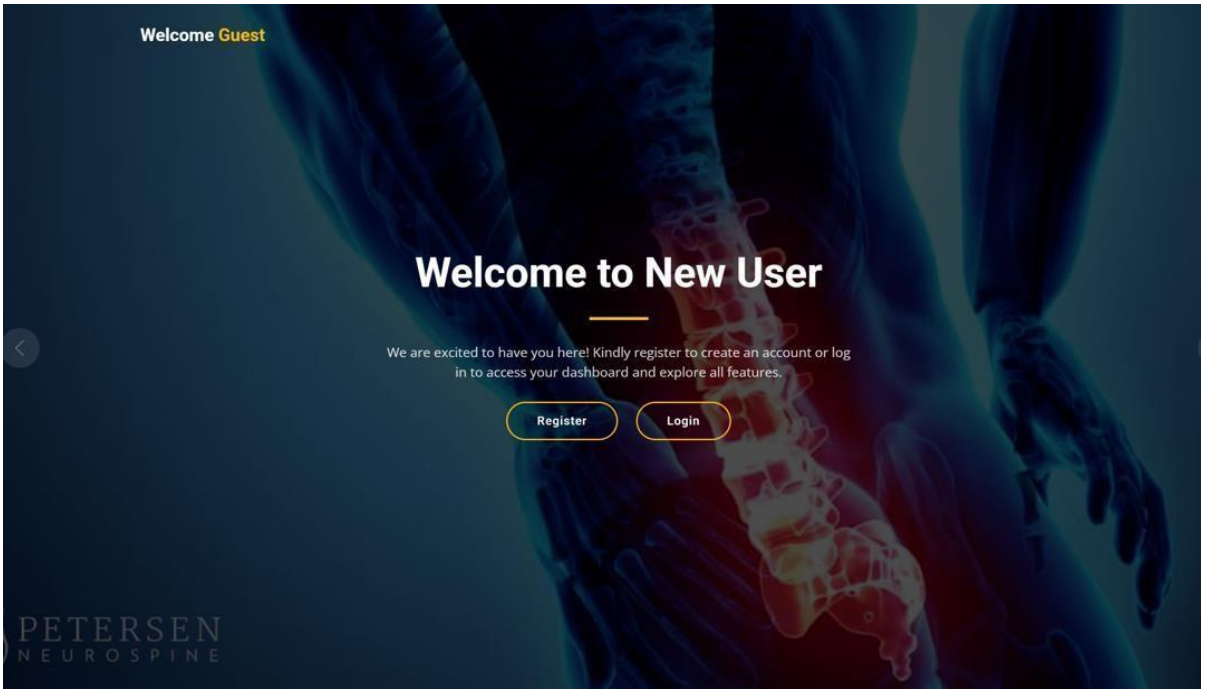


Fig. B.1 Home Page

Registration Page

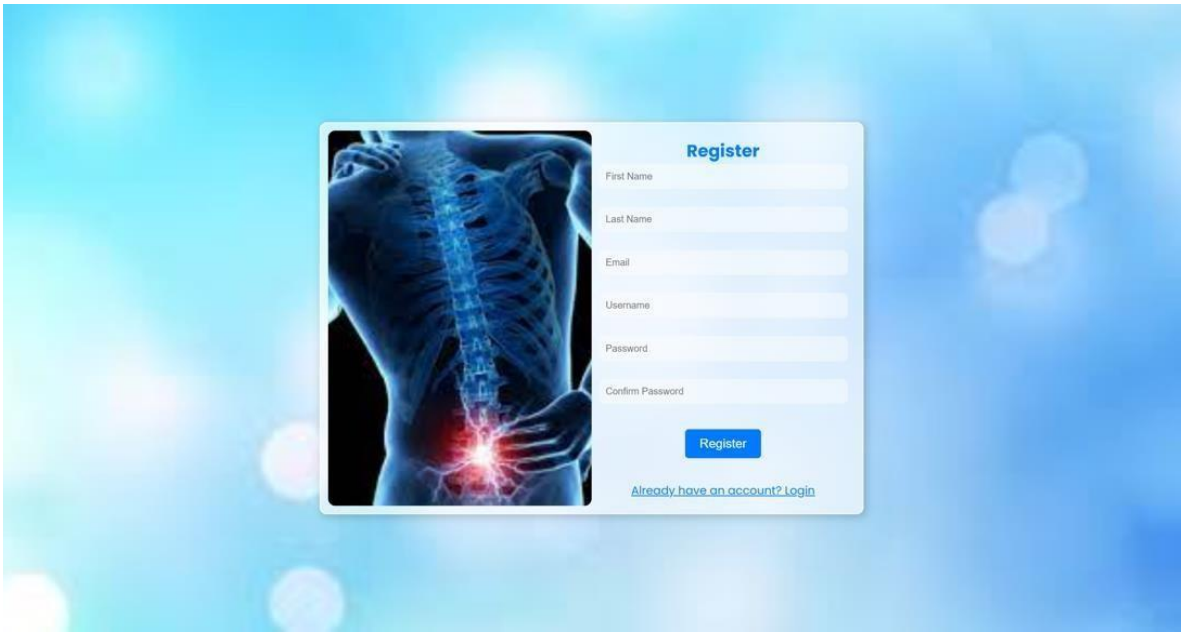


Fig. B.2 Registration Page

`

Profile page

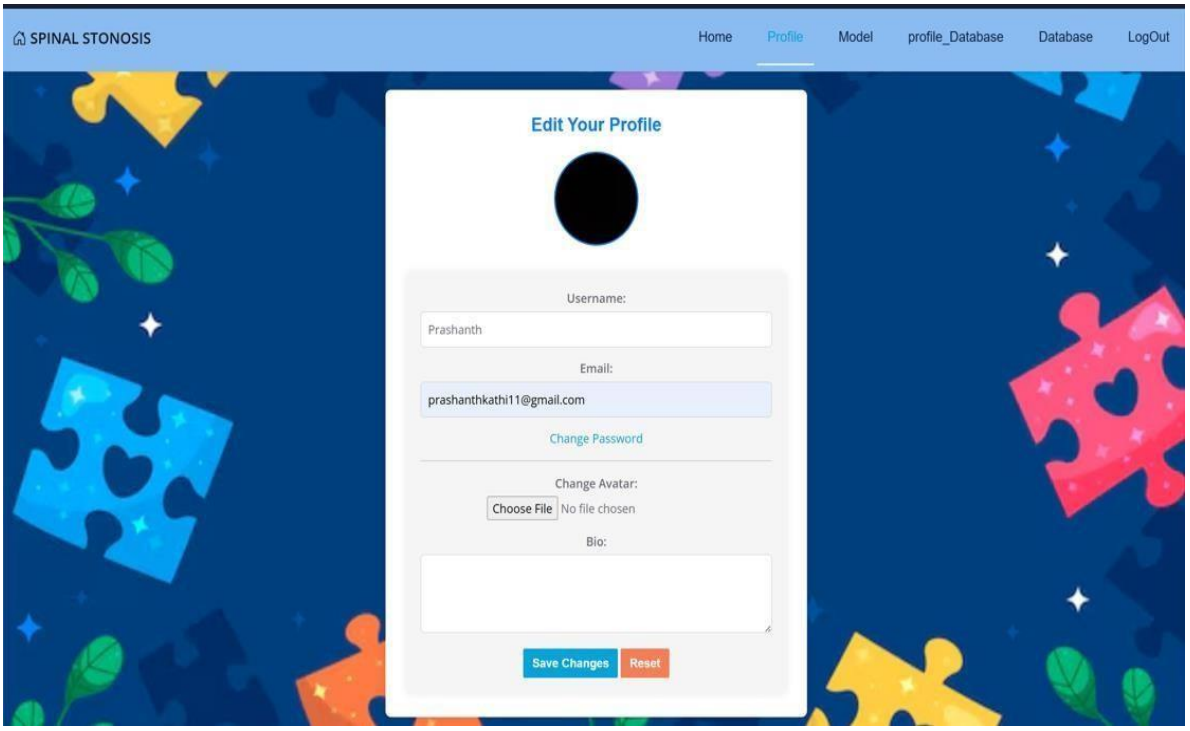


Fig. B.3 Profile Page

Login Page

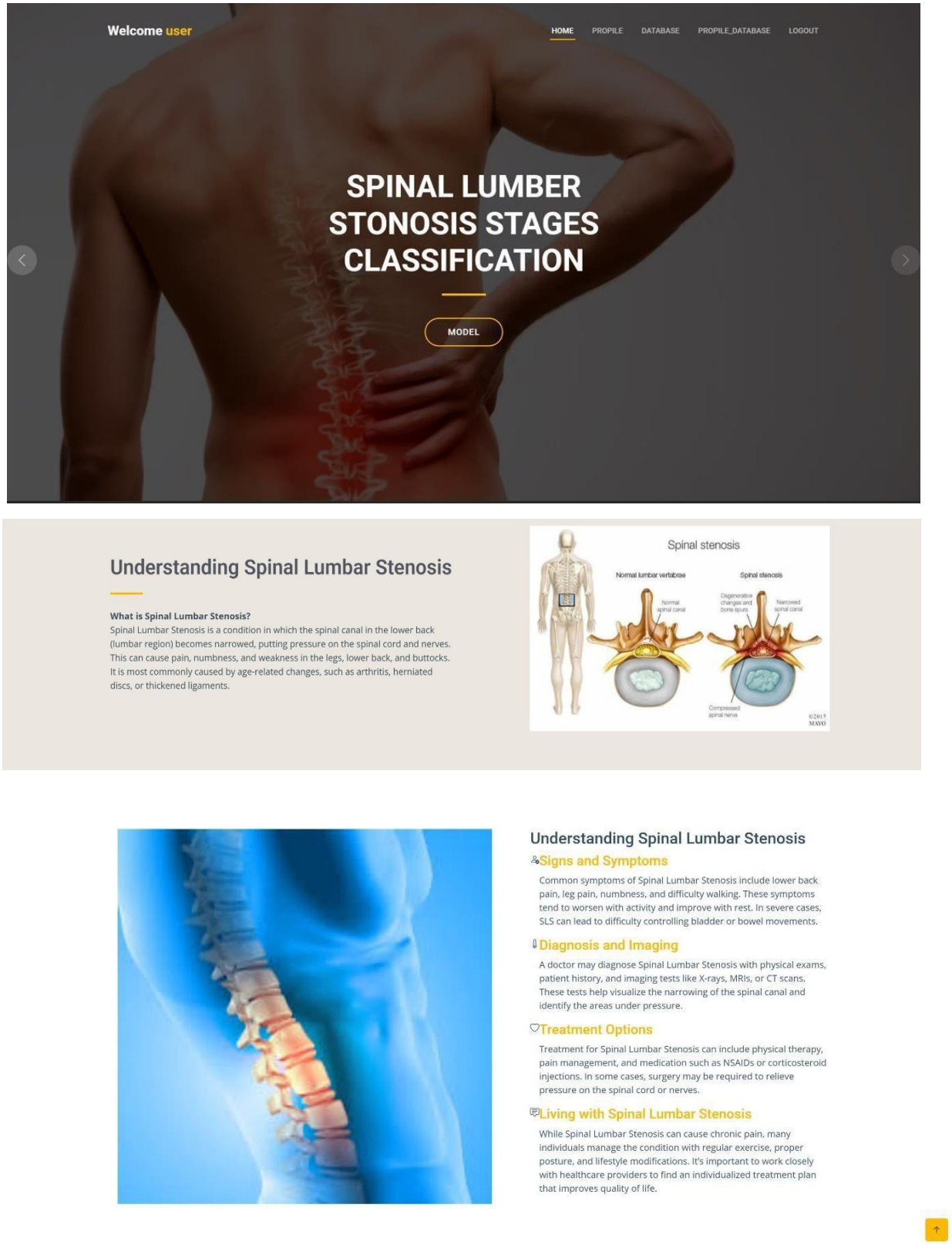


Fig. B.4 Login Page

Data Input Page

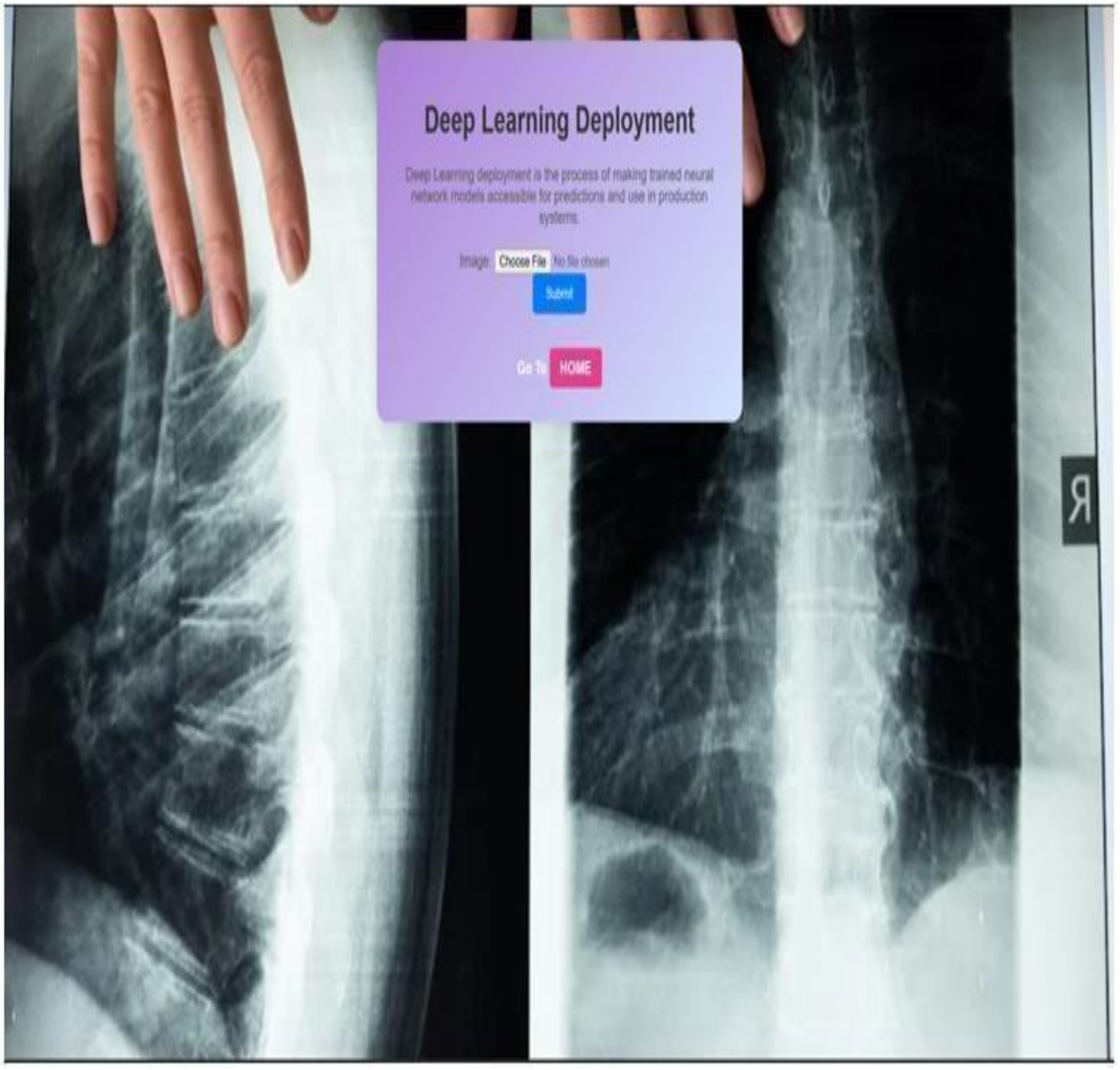
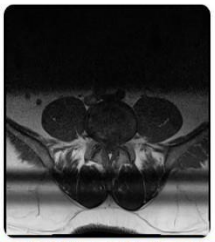


Fig. B.5 Data Input page

Result Page



FOUND THIS IMAGE IS SPINAL LUMBAR STENOSIS (MILD LEVEL)

Description

Spinal lumbar stenosis is a condition where the spinal canal narrows in the lower back, putting pressure on the nerves. In mild cases, symptoms might be subtle, causing occasional discomfort or pain in the lower back or legs.

Causes

Common causes of lumbar stenosis include age-related changes such as osteoarthritis, herniated discs, thickened ligaments, and bone spurs. In some cases, genetics may also play a role.

Prevention

While spinal lumbar stenosis cannot always be prevented, maintaining a healthy weight, exercising regularly, and avoiding activities that strain the back may help reduce the risk of developing the condition.

Treatment

For mild lumbar stenosis, treatments may include physical therapy, pain relievers, and anti-inflammatory medications. In some cases, corticosteroid injections can be used to reduce inflammation and relieve pain.

Diet

There is no specific diet for lumbar stenosis, but a diet rich in calcium and vitamin D can help maintain bone health. Adequate hydration and anti-inflammatory foods may also help manage symptoms.

Physical Activity

Low-impact activities such as walking, swimming, or cycling are beneficial. These activities help strengthen the muscles supporting the spine, improving mobility and reducing pain.

Body Condition Level

Maintaining a healthy body weight is crucial, as extra weight can put additional strain on the spine. Regular exercise to strengthen the core and back muscles is important for managing mild lumbar stenosis.

Affected Age Group

Spinal lumbar stenosis typically affects individuals over the age of 50. However, it can occur at younger ages due to congenital factors or spinal injuries.

Additional Information

Although spinal lumbar stenosis cannot always be reversed, proper management and treatment can help alleviate symptoms and improve quality of life. Surgery is usually considered only for severe cases that do not respond to conservative treatments.

Fig. B.6 Result page

Database Page

Fig. B.7 Database page

`

C. PAPER PUBLICATION

Detection of Spinal Lumbar Stenosis Stages Classification By CNN Techniques

Kathi Prashanth [1]
Department of Computer Science and Engineering
Sathyambama University, Jeppiaar Nagar, Rajiv Gandhi Salai, Chennai – 600119
prashanthkathi11@gmail.com

Kathiravan K[2]
Department of Computer Science and Engineering
Sathyambama University, Jeppiaar Nagar, Rajiv Gandhi Salai, Chennai – 600119
kathiravankumartvm@gmail.com

Mrs. T. Bhanu Shree. B.Tech., M.E., (PhD).,[3]
Assistant Professor
Department of Computer Science and Engineering
Sathyambama University, Jeppiaar Nagar, Rajiv Gandhi Salai, Chennai – 600119
bhanushree.t.cse@sathyabama.ac.in

ABSTRACT- Lumbar spinal stenosis is a widespread degenerative disorder that affects the spinal canal, resulting in neural compression and causing symptoms such as pain and restricted movement. The prompt and precise classification of stenosis stages is essential for effective treatment decisions and comprehensive patient care. This research proposes an advanced method utilizing Convolutional Neural Networks (CNNs) to automatically detect and categorize spinal lumbar stenosis stages. The developed CNN model employs state-of-the-art medical imaging analysis and deep learning techniques to examine MRI and CT scans. The convolutional layers are structured to autonomously extract crucial image features, recognizing subtle differences associated with various stenosis stages. The model is trained on a diverse set of cases representing multiple severity levels, ensuring strong adaptability for new and unseen cases. This study prioritizes achieving exceptional classification accuracy, sensitivity, and specificity, enabling healthcare professionals to make precise treatment plans. The CNN-based system's results are meticulously compared with traditional classification approaches to demonstrate its diagnostic efficiency and superiority. This research plays a vital role in advancing medical imaging technology while fulfilling the pressing need for reliable and rapid lumbar stenosis classification systems. Integrating deep learning in this area enhances diagnostic capabilities and streamlines therapeutic interventions, ultimately leading to better outcomes for patients with spinal lumbar stenosis.

KEYWORDS- Lumbar spinal stenosis, Convolutional Neural Networks (CNNs), deep learning, medical imaging analysis, MRI, CT scans, neural compression, classification accuracy, diagnostic efficiency, treatment planning, healthcare technology.

I. INTRODUCTION

Evaluating and classifying the stages of spinal lumbar stenosis is a significant focus in medical imaging, where misdiagnosis can lead to ineffective treatment. This research leverages Convolutional Neural Networks (CNNs), a crucial element of deep learning, to increase the precision and speed of stenosis detection. By applying CNNs' advanced feature-learning techniques, the proposed solution automates the classification of stenosis severity from medical imaging data. This progressive framework refines the diagnostic process and plays a crucial role in patient care by ensuring early and accurate treatment measures tailored to individual stenosis severity levels.

The diagnosis and classification of spinal lumbar stenosis stages represent a critical challenge in medical imaging, where precision is vital for guiding treatment decisions. Spinal lumbar stenosis, a degenerative condition that results in spinal canal narrowing, can significantly affect patients' mobility and quality of life. Given the complexities of manual image interpretation, there is an increasing demand for automated solutions that can enhance diagnostic accuracy while reducing reliance on subjective human assessment.

This study employs Convolutional Neural Networks (CNNs), an advanced deep learning technique, to develop an automated framework for spinal lumbar stenosis classification. CNNs are highly effective in medical imaging applications due to their capability to learn spatial hierarchies of features from raw image data. The proposed model utilizes MRI and CT scans, with convolutional layers extracting meaningful features that distinguish different stenosis severity levels.

A well-curated dataset comprising labeled stenosis cases is used to train the model, ensuring robustness and adaptability to unseen medical images. Evaluation is performed using standard classification metrics such as accuracy, sensitivity, specificity, and the area under the receiver operating characteristic (ROC) curve. Comparative analysis against conventional diagnostic methods highlights the superior performance of CNN-based classification in terms of speed and reliability.

The integration of deep learning in medical diagnostics is a transformative step toward precision medicine. By automating stenosis classification, this research aims to provide radiologists and clinicians with a powerful AI-driven tool that enhances diagnostic confidence, minimizes errors, and ensures early intervention for better patient care.

II. LITERATURE SURVEY

In paper [1], Jain et al. analyze the incidence and trends of traumatic spinal cord injury (TSCI) in the United States from 1993 to 2012. Their research evaluates demographic

data, injury causes, and hospitalization statistics to provide insights into the epidemiology of TSCI. The study highlights the increasing incidence of non-traumatic spinal cord injuries and discusses factors affecting patient recovery and long-term outcomes.

In paper [2], Harkema et al. examine the impact of epidural stimulation on voluntary movement, assisted stepping, and standing in patients with motor-complete paraplegia. Their case study reveals how electrical stimulation of the lumbosacral spinal cord can facilitate partial recovery in paralyzed individuals. The authors emphasize the role of neuromodulation in reactivating spinal circuits despite severe motor impairment.

In paper [3], Angeli et al. explore how modulating spinal cord excitability enables voluntary movement in individuals with chronic complete paralysis. Their study provides evidence that residual neural pathways remain functional despite a complete spinal lesion. The authors highlight how external stimulation can activate these pathways, allowing voluntary movement to be regained.

In paper [4], Gad et al. investigate how epidural stimulation can initiate bladder voiding in paralyzed rats trained for stepping. Their study reveals that targeted spinal stimulation not only aids locomotion but also influences autonomic functions such as bladder control. The findings suggest potential applications for improving urinary function in spinal cord injury patients.

In paper [5], Gerasimenko et al. investigate the combined effect of epidural spinal cord stimulation and quipazine administration on stepping ability in fully paralyzed adult rats. Their study finds that this combination facilitates stepping-like movements, demonstrating the role of pharmacological agents in enhancing neuromodulatory interventions for spinal cord injury rehabilitation.

In paper [6], Lavrov et al. explore how epidural stimulation modulates spinal locomotor networks in adult rats. Their findings suggest that targeted electrical stimulation can alter neural network dynamics, facilitating movement in animals with spinal cord injuries. This study advances understanding of spinal cord plasticity and motor recovery.

In paper [7], Courtine et al. explore how disconnected spinal circuits can be reactivated and transformed into functional states following the loss of brain input. Their study suggests that spinal cord rehabilitation strategies can harness neuroplasticity to restore movement.

In paper [8], Herman et al. examine how spinal cord stimulation facilitates functional walking in individuals with incomplete spinal cord injuries. Their research finds that neuromodulation improves lower-limb control and enhances walking ability in SCI patients.

In their study [9], Ellaway et al. evaluated different physiological measures for motor function assessment in spinal cord injury cases, emphasizing the use of transcranial magnetic stimulation and spinal reflex analysis. Their review, published in *J. Rehabil. Res. Develop.*, offers valuable insights into post-injury neural activity.

In their research [10], Edwards et al. identified that corticospinal conduction persists in individuals with spinal cord injuries, although voluntary movement is compromised. The study, which appears in *Spinal Cord*, highlights the continued functionality of certain neural pathways despite the loss of motor control.

III. EXISTING SYSTEM

Spinal cord injuries (SCI) frequently result in locomotion dysfunction, greatly reducing the affected individuals' overall well-being. Though no definitive solution currently exists, lower extremity exoskeletons (LEEs) offer a potential means of restoring movement. Over the last two decades, research on LEEs has grown exponentially; however, no comprehensive analysis summarizing these findings has been conducted. This study aims to bridge this gap by performing a systematic review from both an engineering and clinical viewpoint, using systematic review and meta-analysis techniques. The study examines various factors, including structural design, actuation mechanisms, control strategies, and medical assessments. A close examination of past research highlights several critical challenges, including actuator efficiency, motor optimization, motion transition handling, trajectory tracking, system flexibility, and medical feasibility. However, prior studies exhibited multiple limitations: they neglected the deployment stage, implemented only basic techniques, did not apply CNNs or ANNs, omitted the image preprocessing phase, failed to conduct data visualization through graphs and plots, and did not compare diverse architectures to improve accuracy. Additionally, most research solely concentrated on SCI without effectively training datasets to optimize computational performance.

IV. PROPOSED SYSTEM

An advanced system has been developed to automatically detect and classify spinal lumbar stenosis stages using Convolutional Neural Network (CNN) technology, significantly improving diagnostic precision and patient treatment. This deep learning-based approach enables accurate categorization of lumbar stenosis severity, providing healthcare professionals with crucial insights for customized treatment strategies. The CNN model processes MRI and CT scan images, allowing for quick and automated assessment of spinal conditions. By utilizing neural network capabilities, the system enhances spinal health evaluations, promoting early diagnosis and individualized care. This breakthrough significantly elevates diagnostic methodologies in orthopedics, improving overall patient outcomes. To ensure smooth deployment, we utilized frontend technologies like HTML, CSS, Bootstrap, and backend frameworks such as Django or Flask. The image

preprocessing pipeline integrates rescaling, horizontal flipping, shear range, and zoom augmentation to improve image clarity. Additionally, data analysis was performed using graphical representations like plots and graphs. To enhance system performance, multiple CNN architectures were evaluated, leading to improved accuracy. The system successfully classifies four different types of spinal stenosis. Deep learning methods were used to extract and interpret complex patterns in spinal images. We meticulously assessed performance metrics, optimized accuracy, and ensured an efficient neural network implementation. Both CNN and ANN models were executed correctly, with proper dataset training and validation for reliable testing outcomes.

DESIGN ARCHITECTURE

Design is a significant engineering depiction of an intended system. In software development, design is a vital phase where raw requirements are methodically refined into a fully operational software product. It establishes a well-defined structure that provides insights into data frameworks, architectural patterns, user experience components, and integral modules required to bring the system to life.
."

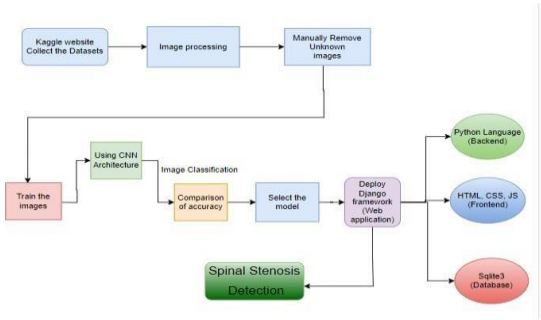


Fig. 1. System Architecture

V. METHODOLOGY

Before training begins, the dataset undergoes preprocessing steps such as image resizing, reshaping, and transformation into an array format. Similar transformations are applied to test images to maintain uniformity. The dataset comprises four spine categories, from which any image can be selected as a test case. A CNN model is trained using this dataset to identify different spinal conditions based on test images. The CNN architecture includes multiple essential layers such as Dense, Dropout, Activation, Flatten, Convolution2D, and MaxPooling2D. After successful training, the model can classify spine images and accurately detect spinal stenosis. A comparison between the trained model and test images determines the final classification.
In a 2D convolution operation, a kernel (a small matrix containing numerical weights) moves across the input data, performing element-wise multiplications and summing the values to generate a single pixel output. This process is systematically repeated, converting input features into new output matrices. The output features result from weighted combinations of the input values, with the kernel's size influencing the number of input features contributing to the output representation.

Unlike fully connected layers, convolutional layers optimize computational efficiency by restricting how output features interact with input data. In a standard fully connected layer, 25×9 = 225 parameters would be necessary to process all input features. However, convolutional layers reduce this requirement to just 9 parameters by limiting each output feature's view to a localized area of input features.
The MaxPooling2D layer compresses spatial information by extracting the highest values from defined windows (pool_size) across input channels. These windows shift (strides), leading to a reduced feature map size. When "valid" padding is applied, output_shape = floor((input_shape - pool_size) / strides) + 1, whereas with "same" padding, output_shape = floor((input_shape - 1) / strides) + 1.
Flatten transforms multi-dimensional feature maps into a single vector for input into fully connected layers. The Dense layer executes output = activation(dot(input, kernel) + bias), where the kernel represents weight values. When processing multi-dimensional inputs, tf.tensordot handles dot product computations.
Dropout mitigates overfitting by randomly disabling a fraction of neurons, redistributing the learning process across the network. The remaining neurons are adjusted using 1/(1 - rate). Image augmentation via the Image Data Generator improves generalization through transformations like rescaling, shearing, zooming, and flipping, ensuring the model remains robust across variations in input data.

VI. ALGORITHM

1) MANUAL ARCHITECTURE

A Convolutional Neural Network (CNN) is a deep learning model specifically designed for processing image data. It assigns importance through learnable weights and biases to different elements in an image, enabling automatic feature extraction and differentiation. Unlike traditional classification methods that require manually designed filters, CNNs learn these features during training. The design of CNNs is inspired by the structure of the human visual cortex, where neurons respond to specific regions of an image known as receptive fields.

The architecture of a CNN consists of several key layers. The **Input Layer** processes images in the form of three-dimensional matrices, which must be converted into a single-column format before entering the network. For instance, a 28×28 pixel image (784 pixels in total) is reshaped into a 784×1 array. The **Convolutional Layer** is the primary feature extraction stage, where filters are applied to detect patterns in the image. This involves computing the dot product between filters and the receptive field, generating feature maps. Following this, the **Pooling Layer** reduces the spatial dimensions of these feature maps to enhance computational efficiency. A common technique is max pooling with a stride of 2, which reduces a 4×4 input

to 2×2 dimensions.Once the essential features are extracted, the **Fully Connected Layer (FC)** links neurons from previous layers to classify the identified patterns. The **Softmax/Logistic Layer** is responsible for classification, where logistic regression is used for binary classification problems, while softmax is employed for multi-class classification. Finally, the **Output Layer** generates the prediction in the form of a one-hot encoded label, indicating the recognized class of the image. This structured approach enables CNNs to efficiently perform complex image recognition and classification tasks.

2) GOOGLE NET

GoogleNet, commonly called Inception-v1, is a deep neural network designed for image classification tasks. Developed by Google's research team under Christian Szegedy, it won first place in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2014. The model's major contribution is the introduction of the "Inception module," which performs convolutions with different filter sizes in parallel, allowing the network to recognize both detailed and abstract features effectively.The Inception module combines 1x1, 3x3, and 5x5 convolutions alongside max-pooling to analyze features at multiple scales. The 1x1 convolution reduces dimensionality, optimizing processing power without sacrificing accuracy.GoogleNet consists of multiple Inception modules stacked on top of each other. As the depth increases, the model learns more complex representations. The simultaneous use of different convolution sizes improves computational efficiency.To mitigate vanishing gradients, GoogleNet incorporates auxiliary classifiers within the intermediate layers. These additional classifiers enhance gradient flow and contribute to better training stability.Rather than using fully connected layers, GoogleNet applies global average pooling, which averages each feature map in the final convolutional layer to generate a classification-ready vector. This approach reduces overfitting and decreases parameter count. GoogleNet set a new benchmark in deep learning, proving the potential of deep architectures in image recognition. It not only achieved state-of-the-art accuracy in ILSVRC 2014 but also maintained high efficiency. Subsequent versions like Inception-v2, Inception-v3, and Inception-v4 further refined its capabilities.

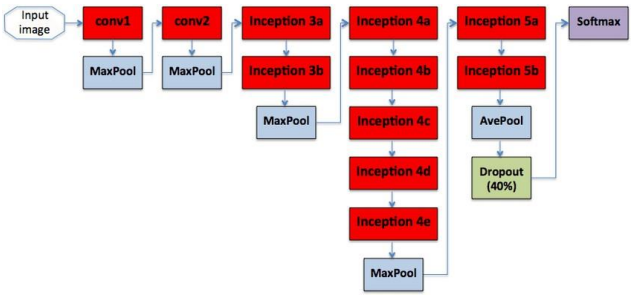


Fig. 2. GoogleNet model

Fig. 3. Google Net Architecture

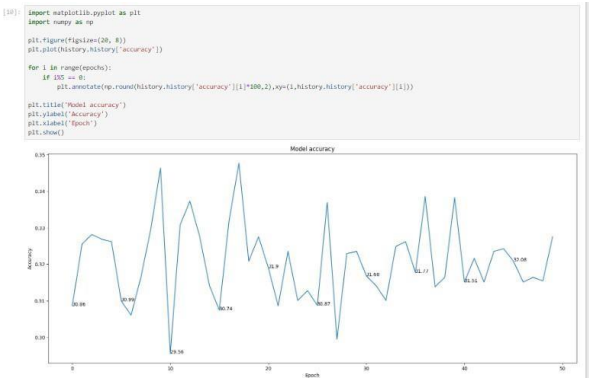


Fig. 4. Google Net Architecture Model Accuracy

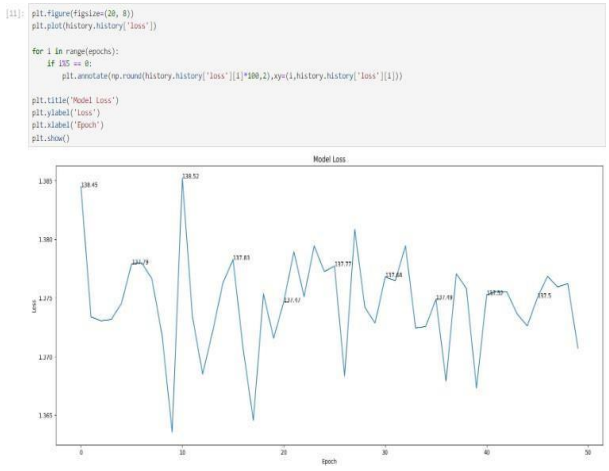


Fig. 5. Google Net Architecture Model Loss

3) LENET

LeNet, introduced by Yann LeCun and his collaborators in the early 1990s, is one of the first convolutional neural networks (CNNs) ever developed. It was designed to process handwritten digit recognition tasks, such as analyzing bank checks and postal codes. LeNet set the stage for modern CNN-based models and helped drive advancements in computer vision and deep learning.The input layer accepts grayscale images, usually representing handwritten digits, with dimensions of 32x32 pixels.Two convolutional layers are used in LeNet to extract essential features from the input images. These layers contain filters that slide across the image to detect edges, shapes, and patterns, forming feature maps. Pooling layers follow each convolutional layer, reducing the spatial size of the feature

maps while maintaining crucial details. This optimization helps improve efficiency and reduce computational overhead.The pooled feature maps are flattened and sent through fully connected layers, which process the information and classify the images accordingly. While LeNet is simpler compared to modern deep learning architectures like VGG or ResNet, its innovative use of convolution and pooling layers made it a foundation for future CNN models. LeNet's success in digit recognition contributed to the revival of neural networks in the early 2000s, paving the way for more sophisticated image classification techniques.

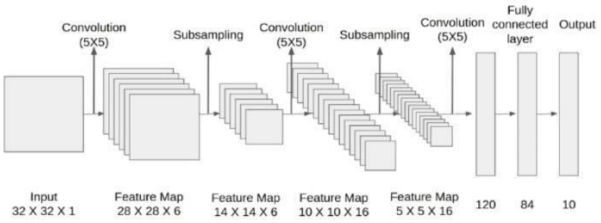


Fig. 6. Lenet Architecture

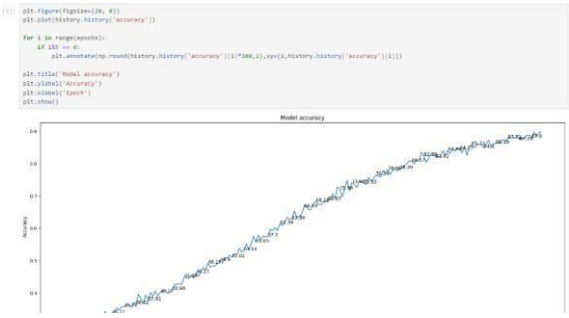


Fig. 7. Lenet Model Accuracy

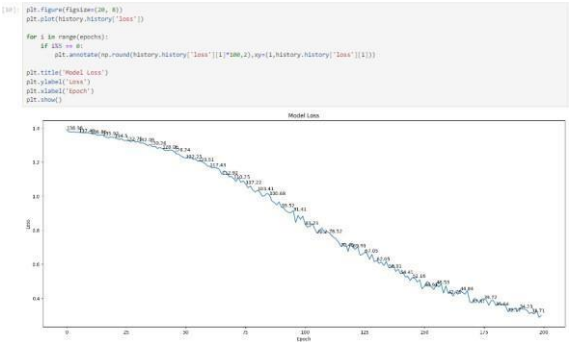


Fig. 8. Lenet Model Loss

4) *SPINE STENOSIS CLASSIFICATION IDENTIFICATION*

Keras preprocessing is utilized to process input images, which are subsequently converted into arrays via the Pillow library and the image_to_array function. This step facilitates spine stenosis classification. A specific detection function is used to analyze and detect spine stenosis cases. The classification framework operates on a dual-channel convolutional neural network, ensuring accurate recognition of spine stenosis types. The processed images are fed into the Inception layer of the CNN. The training process involves feature extraction and classification using convolutional layers, refining the model's ability to identify and categorize spine stenosis effectively.

5) *RESNET ARCHITECTURE*

ResNet, or Residual Network, was developed by Microsoft Research in 2015 to tackle the issue of vanishing gradients in deep neural networks. It incorporates residual learning with shortcut connections, allowing information to bypass one or more layers and be directly added to later outputs, thereby ensuring effective gradient transmission. Unlike traditional deep networks that degrade in accuracy as they grow deeper, ResNet makes it possible to train significantly deep models without loss of performance. It consists of residual blocks (seen in ResNet-18 and ResNet-34), which feature two 3×3 convolutional layers, and bottleneck blocks (used in ResNet-50, ResNet-101, and ResNet-152), which include 1×1 convolutions for improved efficiency and computational reduction. ResNet's key strength lies in its ability to simplify the learning process by focusing on residual functions rather than complete transformations, making optimization more straightforward. It has redefined computer vision tasks such as image classification, object detection, and segmentation, excelling on large datasets like ImageNet. Moreover, ResNet has laid the foundation for newer architectures like ResNeXt, EfficientNet, and Vision Transformers. Because of its accuracy, efficiency, and scalability, ResNet continues to be widely utilized in modern AI-based applications.

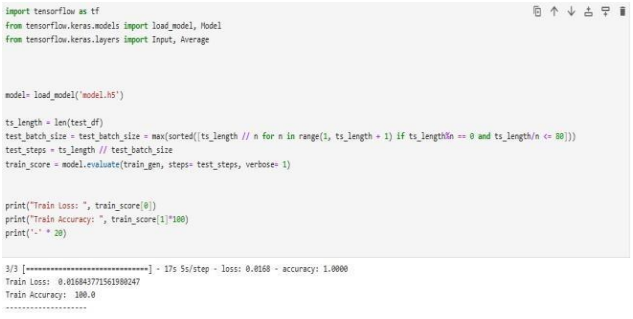


Fig. 9. Resnet Runtime Accuracy

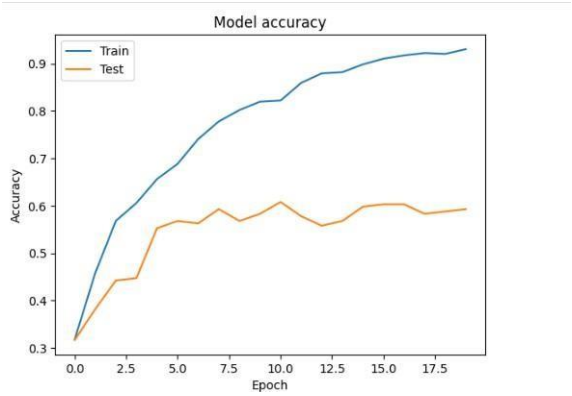


Fig. 10. Model Accuracy Graph

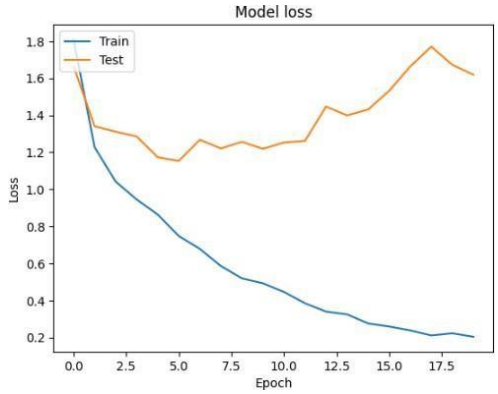


Fig. 11. Model Loss Graph

VII. RESULTS

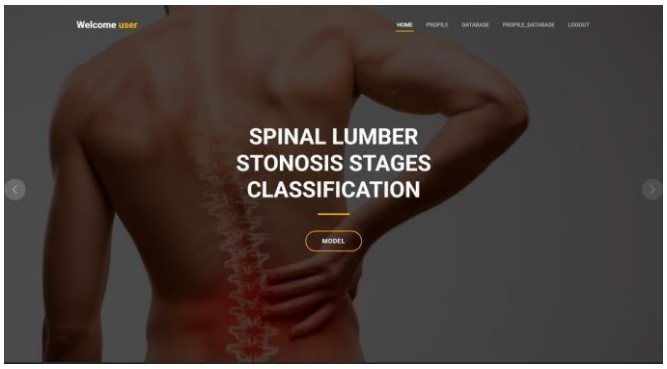


Fig. 12. Login Page

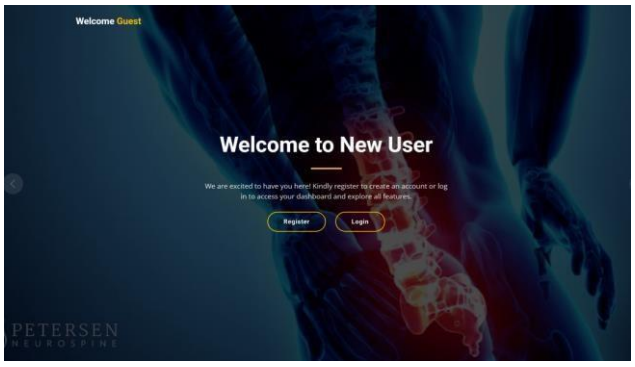


Fig. 13. Welcome Page

Fig. 14. Registeration Page

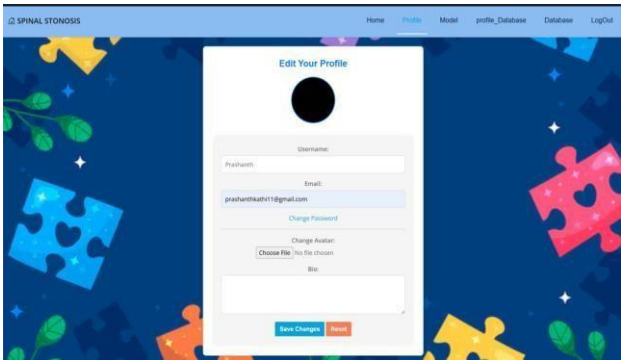


Fig. 15. Profile Page

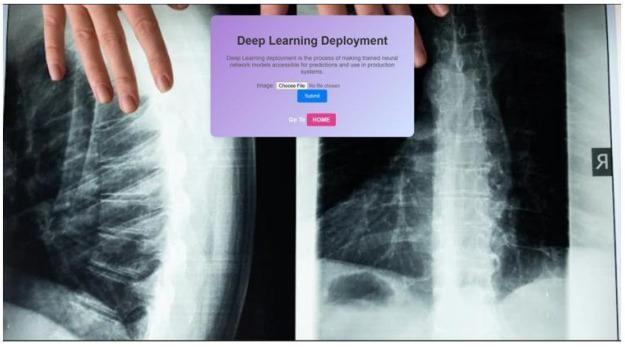


Fig. 16. MRI Scan Upload Page

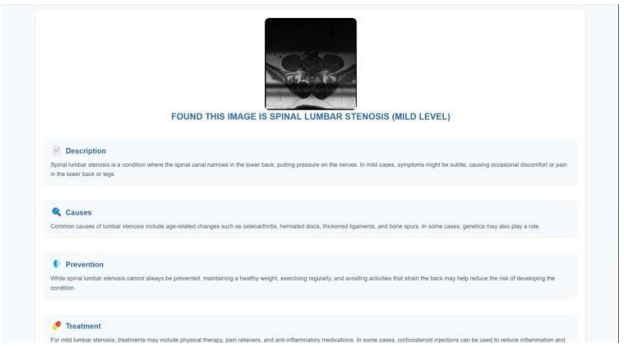


Fig. 17. Result Page

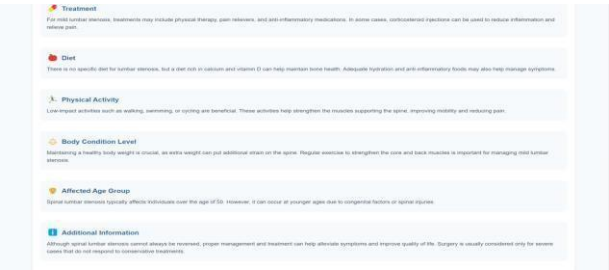


Fig. 18. Discussion Page

Fig. 19. Database Page

VIII. CONCLUSION

Ultimately, the utilization of CNN architectures in detecting spinal stenosis has revolutionized musculoskeletal diagnostics by improving precision and efficiency. The ability of CNNs to extract and analyze intricate patterns within medical images has greatly enhanced the classification of spinal conditions. This advancement paves the way for more streamlined diagnostic procedures and improved clinical insights. As AI-driven imaging continues to evolve, CNN-based models are expected to further enhance spine healthcare, ensuring more accurate and timely diagnoses. Continuous research and development will unlock even greater potential in automated musculoskeletal imaging analysis.

IX. FUTURE WORK

To enhance the efficiency and accuracy of spinal stenosis diagnosis, the medical sector is automating the eligibility assessment process in real-time. By deploying this system in a cloud ecosystem, healthcare professionals will benefit from scalable computing power, remote data access, and improved security. AI-powered diagnostic tools will facilitate quicker detection of spinal conditions, reducing delays in medical evaluations. This cloud integration will allow seamless coordination between specialists, ensuring faster and more precise medical decision-making while optimizing the overall diagnostic workflow.

REFERENCES

[1] N. B. Jain et al., “Traumatic spinal cord injury in the United States, 1993– 2012,” JAMA, vol. 313, no. 22, pp. 2236–2243, 2015.

[2] S. J. Harkema et al., “Effect of epidural stimulation of the lumbosacral spinal cord on voluntary movement, standing, and assisted stepping after motor complete paraplegia: A case study,” Lancet, vol. 377, pp. 1938– 1947, 2011.

[3] C. A. Angeli et al., “Altering spinal cord excitability enables voluntary movements after chronic complete paralysis in humans,” Brain, vol. 137, pp. 1394–1409, Apr. 2014.

[4] P. N. Gad et al., “Initiation of bladder voiding with epidural stimulation in paralyzed, step trained rats,” PLOS ONE, vol. 9, no. 9, pp. 1–9, 2014.

[5] Y. P. Gerasimenko et al., “Epidural spinal cord stimulation plus quipazine administration enable stepping in complete spinal adult rats,” J. Neurophysiol., vol. 98, no. 5, pp. 2525–2536, 2007.

[6] I. Lavrov et al., “Epidural stimulation induced modulation of spinal locomotor networks in adult spinal rats,” J. Neurosci., vol. 28, no. 23, pp. 6022– 6029, 2008.

[7] G. Courtine et al., “Transformation of nonfunctional spinal circuits into functional states after the loss of brain input,” Neuroscience, vol. 12, no. 10, pp. 1333–1342, 2009.

[8] R. Herman et al., “Spinal cord stimulation facilitates functional walking in a chronic, incomplete spinal cord injured,” Spinal Cord, vol. 40, no. 2, pp. 65–68, 2002.

[9] [9] P. H. Ellaway et al., “Review of physiological motor outcome measures in spinal cord injury using transcranial magnetic stimulation and spinal reflexes,” J. Rehabil. Res. Develop., vol. 44, pp. 69–76, 2007.

[10] D. J. Edwards et al., “Preserved corticospinal conduction without voluntary movement after spinal cord injury,” Spinal Cord, vol. 51, pp. 765–767, 2013.

[11] Kalichman L, Cole R, Kim DH, Li L, Suri P, Guermazi A, Hunter DJ (2009) Spinal stenosis prevalence and association with symptoms: the Framingham Study. Spine J 9(7):545–550 - DOI - PubMed - PMC

[12] Wu A-M, Zou F, Cao Y, Xia D-D, He W, Zhu B, Chen D, Ni W-F, Wang X-Y, Kwan K (2017) Lumbar spinal stenosis: an update on the epidemiology, diagnosis and treatment. AME Med J 2(5)

[13] Katz JN, Harris MB (2008) Lumbar spinal stenosis. N Engl J Med 358(8):818–825 - DOI - PubMed

[14] Pannell WC, Savin DD, Scott TP, Wang JC, Daubs MD (2015) Trends in the surgical treatment of lumbar spine disease in the United States. Spine J 15(8):1719–1727 - DOI - PubMed

[15] Lurie J, Tomkins-Lane C (2016) Management of lumbar spinal stenosis. BMJ 352:h6234 - DOI - PubMed - PMC

`

PLAGIARISM REPORT

RESEARCH PAPER (1)-PLAG.doc

by Turnitin Account

Submission date: 21-Mar-2025 04:52PM (UTC+0900)
Submission ID: 2615806019
File name: RESEARCH_PAPER_1_-PLAG.doc (3.42M)
Word count: 3084
Character count: 19607

`

RESEARCH PAPER (1)-PLAG.doc

ORIGINALITY REPORT

8%	4%	3%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	cran.rstudio.com Internet Source	1%
2	www.geeksforgeeks.org Internet Source	1%
3	Submitted to Siksha 'O' Anusandhan University Student Paper	1%
4	www.jetir.org Internet Source	1%
5	Submitted to Vaasan yliopisto Student Paper	<1%
6	Submitted to AlHussein Technical University Student Paper	<1%
7	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1%
8	Submitted to Crown Institute of Business and Technology Student Paper	<1%
9	www.mdpi.com Internet Source	<1%
10	Yuandong Lin, Ji Ma, Jun-Hu Cheng, Da-Wen Sun. "Visible detection of chilled beef freshness using a paper-based colourimetric sensor array combining with deep learning algorithms", Food Chemistry, 2024 Publication	<1%

`

	harbinengineeringjournal.com Internet Source	<1%
	Harkema, S.J.. "Plasticity of interneuronal networks of the functionally isolated human spinal cord", Brain Research Reviews, 200801 Publication	<1%
	Jain, Nitin B., Gregory D. Ayers, Emily N. Peterson, Mitchel B. Harris, Leslie Morse, Kevin C. O'Connor, and Eric Garshick. "Traumatic Spinal Cord Injury in the United States, 1993-2012", JAMA, 2015. Publication	<1%
	towardsdatascience.com Internet Source	<1%
15	lirias.kuleuven.be Internet Source	<1%
16	katalog.ub.uni-paderborn.de Internet Source	<1%
17	link.springer.com Internet Source	<1%
18	ufv-inf721-2024-2.lucasnferreira.com Internet Source	<1%
19	Angeli, C. A., V. R. Edgerton, Y. P. Gerasimenko, and S. J. Harkema. "Altering spinal cord excitability enables voluntary movements after chronic complete paralysis in humans", Brain, 2014. Publication	<1%

ACCEPTANCE INTIMATION

Decision: Accept
1 message

Microsoft CMT <email@msr-cmt.org> Fri, Mar 28, 2025 at 23:35
Reply-to: Vineeta Singh <vineetasingh@csjmu.ac.in>
To: KATHI PRASHANTH <prashanthkathi11@gmail.com>
Cc: alok@csjmu.ac.in, ajeetsrivastava@csjmu.ac.in, vineetasingh@csjmu.ac.in

Subject: Acceptance of Your Paper for ICRAECCT 2025

Dear Author(s),
We are pleased to inform you that your paper has been provisionally accepted for presentation at ICRAECCT 2025. You are requested to incorporate reviewer comments to your revised manuscript and resubmit the revised file by 02 April 2025 to the Microsoft CMT portal. Please note that your slot for paper presentation at ICRAECCT 2025 will be confirmed only after paying the registration fees (Registration Deadline : 02 April 2025).

Conference fees Payment link: https://secure.paytmpayments.com/link/paymentForm/23536/LL_724028351

registration form link: https://docs.google.com/forms/d/e/1FAIpQLSeCzsqfL9feuHP44Jlxmpq Y p6SXxiGuRVLopLqmR4ZI4RgDwg/viewform

Your paper may be considered for publication in any one of the following options: Journals, Edited Books, or Conference Proceedings as per the overall quality and content of your submitted manuscript. Detailed Publication Process and related Guidelines along with allied Formatting instructions will be communicated to you shortly via email.
Please Note that: no additional publication fee is required beyond the conference registration fee.
Current Publication Opportunities:
SCIE/Scopus Indexed Journals
1.Selected high-quality extended papers will be invited for publication in special issues of the following Journals:
2.Current Medical Imaging (ISSN / eISSN: 1573-4056 / 1875-6603) - Bentham Science Publisher - SCIE Indexed
3.Current Computer-Aided Drug Design (ISSN / eISSN: 1573-4099 / 1875-6697) - Bentham Science Publisher - SCIE Indexed
4.Franklin Open (Print ISSN: 1879-2693 / 0016-0032) - Elsevier - Scopus Indexed
Scopus Indexed Edited Books
1.Recent Advances in Smart Communication Technologies for a Sustainable Future - IGI Global
2.Ensuring Secure Connectivity Through AI-Powered Wireless Systems - IGI Global
3.Surgical Robotics and Computer Assisted Surgery: Fundamentals and Advances - CRC Press (Taylor & Francis Group)
4.Machine Learning and IoT in Disease Diagnosis and Prediction: Advancing Health Care with AI-Driven Insights - Bentham Science Publisher
5.Neuromorphic Engineering: Transforming Technology with Brain-Inspired Systems - Wiley / Scrivener Publishing
6.AI-Driven Decision Making: Intelligent Solutions for a Data-Driven World - Bentham Science Publisher (Proposal Submitted to Publisher)
Scopus Indexed Conference Proceedings
1.IET Conference Proceedings (Scopus Indexed) (Proposal under review)
ISBN Conference Proceedings
1.Remaining Selected Papers will be published in conference proceedings with ISBN.
We look forward to your active participation and contribution to the success of ICRAECCT 2025.

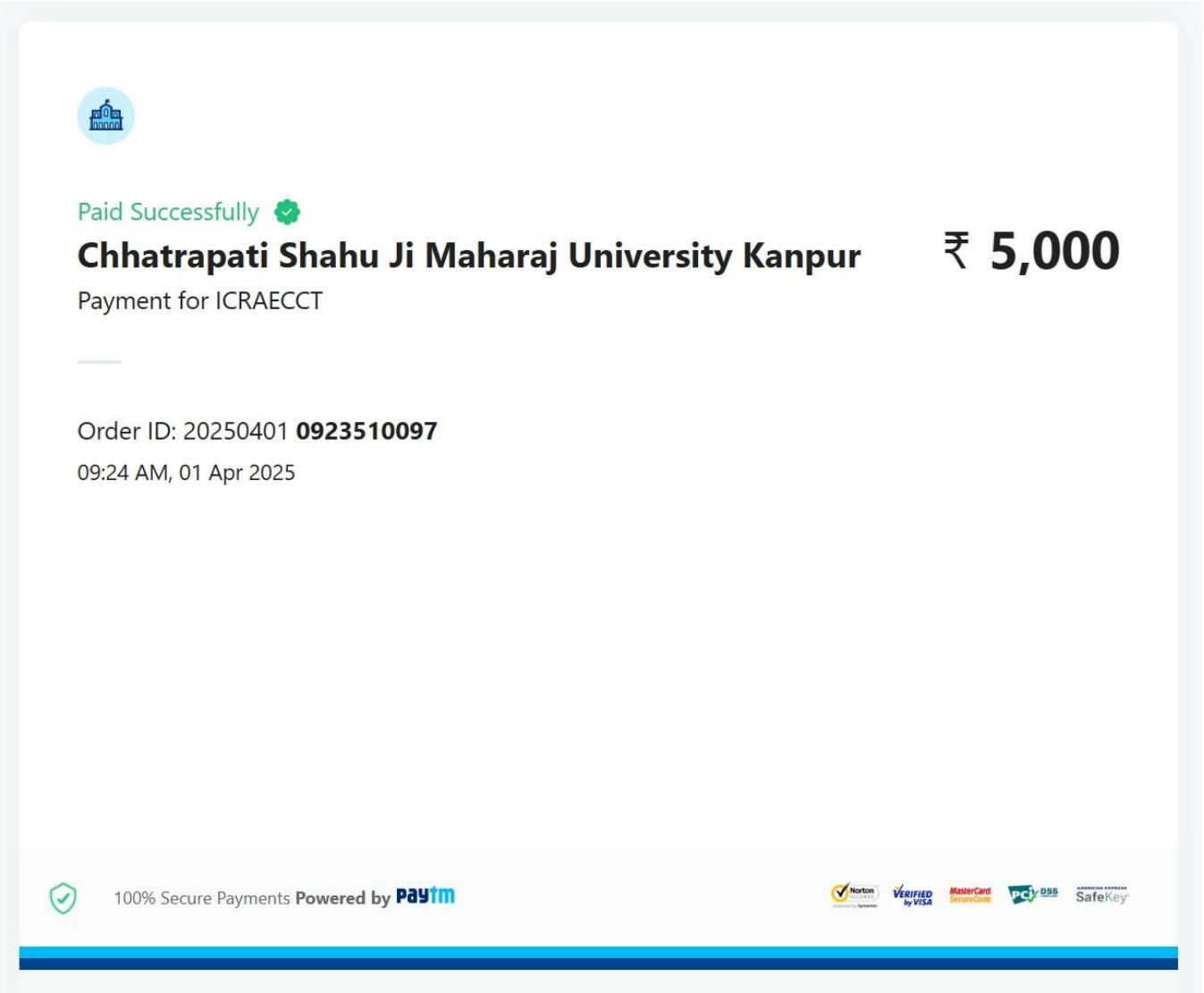
For Further queries you can reach out to us on icraecct@csjmu.ac.in/vineeta.singh.cs@gmail.com/alok@csjmu.ac.in

Warm Regards,
ICRAECCT 2025 TECHNICAL PROGRAMME COMMITTEE
CSJM University, Kanpur

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

`

PAYMENT PROOF SCREENSHOT